Journal of Network and Computer Applications 250 (2026) 104475

Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca

Research paper

MLRan: A behavioural dataset for ransomware analysis and detection

Faithful Chiagoziem Onwuegbuche [a,b,c,*], Sunday Olaoluwa Adelodun [d], Anca Delia Jurcut [a], Liliana Pasquale [a,c]

[a] School of Computer Science, University College Dublin, Dublin, Ireland
[b] SFI Center for Research Training in Machine Learning, Dublin, Ireland
[c] Lero, Research Ireland Centre for Software, Ireland
[d] University of Ibadan, Nigeria

ARTICLE INFO

Dataset link: https://github.com/faithfulco/mlran

Keywords:
Ransomware dataset
Dynamic analysis
Machine learning
Malware
Cybersecurity

ABSTRACT

Ransomware remains a critical threat to cybersecurity, yet publicly available datasets for training machine learning-based ransomware detection models are scarce and often have limited sample size, diversity, and reproducibility. In this paper, we introduce MLRan, a behavioural ransomware dataset, comprising over 4800 samples across 64 ransomware families and a balanced set of goodware samples. The samples span from 2006 to 2024 and encompass the four major types of ransomware: locker, crypto, ransomware-as-a-service, and modern variants. We also propose guidelines (GUIDE-MLRan), inspired by previous work, for constructing high-quality behavioural ransomware datasets, which informed the curation of our dataset. We evaluated the ransomware detection performance of several machine learning (ML) models using MLRan. For this purpose, we performed feature selection by conducting mutual information filtering to reduce the initial 6.4 million features to 24,162, followed by recursive feature elimination, yielding 483 highly informative features. The ML models achieved an accuracy, precision and recall of up to 98.7%, 98.9%, 98.5%, respectively. Using SHAP and LIME, we identified critical indicators of malicious behaviour, including registry tampering, strings, and API misuse. The dataset and source code for feature extraction, selection, ML training, and evaluation are available publicly to support replicability and encourage future research, which can be found at https://github.com/faithfulco/mlran.

1. Introduction

Ransomware attacks continue to pose a significant threat to individuals, organisations, and governments worldwide, causing substantial financial and operational disruptions (Onwuegbuche et al., 2023; Beaman et al., 2021; Jeremiah et al., 2024). Recent public disclosure data suggest that more than 5600 ransomware attacks were reported globally in 2024, with a continued upward trend into 2025 (Office of Intelligence and Analysis, U.S. Department of Homeland Security, 2025). At the organisational level, according to the (CrowdStrike, 2025) State of Ransomware Survey, 78% of organisations surveyed experienced a ransomware attack in the preceding year, yet only 22% recovered within 24 h, highlighting a persistent gap between perceived preparedness and effective organisational resilience. Moreover, ransom payments offer limited protection, as 83% of paying victims suffered subsequent attacks, 93% confirmed data exfiltration despite payment, and 45% were unable to fully recover their data (CrowdStrike, 2025). The cost of ransomware has shifted; average recovery costs, excluding ransoms, reached an estimated $1.53 million in 2025 (Sophos, 2025), while ransom payments themselves have declined in some regions as victims increasingly leverage backups and refuse demands (Kelly, 2025). Critical sectors such as healthcare, energy, and transportation experienced disproportionate targeting, with attacks against such infrastructure increasing by roughly 34% in 2025 (KELA, 2025). These alarming figures not only illustrate why ransomware is now regarded as the foremost cyber threat by 62% of C-suite executives but also highlight the urgent need for advanced detection and mitigation strategies (Zaki, 2023).

Defending against ransomware remains challenging, as traditional signature-based antivirus measures are often insufficient (McIntosh et al., 2021; Kapoor et al., 2021). In response, the cybersecurity community has increasingly turned to machine learning (ML) as a primary tool for ransomware detection and prevention (Al-Hawawreh et al., 2024; Onwuegbuche et al., 2023). ML models can automatically learn complex behavioural patterns from large datasets, enabling them to recognise subtle indicators of ransomware activity—such as mass file encryption or abnormal process behaviour—that may elude human

* Corresponding author at: School of Computer Science, University College Dublin, Dublin, Ireland.
E-mail address: faithful.chiagoziemonwuegb@ucdconnect.ie (F.C. Onwuegbuche).

https://doi.org/10.1016/j.jnca.2026.104475
Received 20 May 2025; Received in revised form 31 January 2026; Accepted 12 March 2026
Available online 14 March 2026
1084-8045/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

analysts or static rule-based systems (Fernando and Komninos, 2024). Unlike signature-based approaches, ML-based methods can adapt to new ransomware types by (re-)training on updated data, keeping pace with the constantly evolving tactics of attackers.

However, the effectiveness of these ML models depends on the quality and representativeness of the data on which they are trained (Herrera-Silva and Hernández-Álvarez, 2023). Well-trained algorithms are only as robust as the datasets that underpin their learning. Most research surveys on ML-based ransomware detection have identified the limited availability of publicly accessible, high-quality datasets as a significant challenge (Ispahany et al., 2024; Beaman et al., 2021; McIntosh et al., 2021; Alraizza and Algarni, 2023). As noted by Urooj et al. (2021), no benchmark dataset currently serves as a reliable foundation for developing and evaluating ransomware detection systems. Several earlier datasets, including those in Sgandurra et al. (2016), Herrera-Silva and Hernández-Álvarez (2023), Jethva et al. (2020), Hirano et al. (2022), Hirano and Kobayashi (2025), are characterised by small sample sizes and limited coverage of ransomware families, which constrains their representativeness and generalisation capability. In addition, many datasets are imbalanced, often containing disproportionately fewer benign samples compared to ransomware or vice versa, thus skewing ML model training (Hou et al., 2024; Continella et al., 2016; Marcinkowski et al., 2024). Furthermore, most datasets capture only a narrow aspect of ransomware behaviour — such as low-level I/O operations, storage and memory access patterns (Continella et al., 2016; Hirano et al., 2022; Hirano and Kobayashi, 2025) — without capturing comprehensive behavioural features. Manual and inconsistent data curation processes further impede reproducibility and the standardisation of experimental conditions, while the lack of public availability hinders future research (Moreira et al., 2024).

Our approach addresses these limitations by proposing MLRan, a large, diverse, balanced behavioural ransomware dataset featuring over 4800 balanced samples of ransomware and goodware. Our dataset focuses on ransomware targeting Windows systems since 95% of the considered ransomware files are Windows-based executables or DLLs.[1] MLRan covers 4 major ransomware types, namely locker, crypto, ransomware-as-a-service (RaaS), and modern variants. It includes 64 ransomware families from 2008 to 2024, and captures nine key behavioural features, including API calls, registry keys, file and directory operations, strings, network activity, system processes, dropped files, and digital signatures.

Inspired by previous work, we propose guidelines (GUIDE-MLRan) for constructing high-quality behavioural ransomware datasets and ensuring their reproducibility. We used these guidelines to ensure the rigorous curation of the MLRan dataset. To streamline the dynamic analysis of malware necessary to curate the dataset, we enhanced the functionality of the Cuckoo Sandbox (an open-source automated malware analysis system) by automating file submission and sorting of analysis results. This extension reduced the manual effort necessary to generate the dataset, ensuring its consistent and efficient development.

We empirically demonstrate that the MLRan dataset is well-suited for building effective ransomware detection models. To reduce data dimensionality, we implemented a novel feature selection strategy based on mutual information filtering to reduce the initial 6.4 million features to 24,162, followed by recursive feature elimination, yielding 483 highly informative features. The ML models achieved an accuracy, precision and recall of up to 98.7%, 98.9%, 98.5%, respectively. Our feature selection strategy reduces data dimensionality and computational cost without compromising performance, highlighting the dataset's practicality and scalability.

We analyse key ransomware behaviours using explainable AI techniques such as SHapley Additive exPlanations (SHAP) and Local Interpretable Model-agnostic Explanations (LIME). This analysis identifies the most discriminative features for distinguishing ransomware from benign software, providing valuable insights for future research on ransomware detection. We made our dataset and tool implementation publicly available[2] to enhance reproducibility and support future research. The tool provides several functionalities, including ransomware and goodware sample collection, automated file processing and results sorting in Cuckoo Sandbox, feature extraction for nine behavioural categories, feature selection using our proposed strategy, and machine learning code, enabling researchers to replicate and extend our work as new ransomware variants emerge. All data sharing follows strict ethical guidelines and excludes raw malware binaries.

In summary, the key contributions of this work are as follows:

- **GUIDE-MLRan**, a principled set of guidelines for constructing high-quality behavioural ransomware datasets, is introduced. The guidelines formalise best practices for dataset collection, curation, labelling, analysis, and reproducibility, addressing long-standing challenges in benchmarking ML-based ransomware detection models.
- **MLRan**, a large-scale, balanced, and publicly available behavioural ransomware dataset, is presented. The dataset comprises nearly 5000 ransomware and benign samples spanning 64 ransomware families (2008–2024) and four major ransomware types, and captures nine complementary behavioural feature categories derived from dynamic analysis.
- **An enhanced Cuckoo Sandbox workflow** is developed to support scalable and reproducible dynamic analysis of samples. The extensions automate sample submission, analysis, and result organisation, substantially reducing manual effort and ensuring consistent behavioural data collection.
- **The effectiveness and practicality of MLRan** are demonstrated through extensive machine learning experiments. A two-stage feature selection strategy reduces an initial feature space of 6.4 million dimensions to a compact and informative subset, enabling accurate and computationally efficient ransomware detection.
- **Behavioural insights** are provided through explainable AI techniques. SHAP and LIME analyses identify the most discriminative ransomware behaviours, supporting model interpretability and informing future detection strategies.
- **All datasets and supporting tools** are released as open-source artefacts. The resources enable full reproducibility and facilitate extension by the research community, while adhering to strict ethical guidelines that exclude raw ransomware binaries.

The rest of the paper is organised as follows. In Section 2, we review existing publicly available behavioural ransomware datasets, their strengths and limitations and motivate the need for the MLRan dataset. In Section 3 we introduce GUIDE-MLRan, our set of guidelines for developing high-quality behavioural ransomware datasets, while in Section 4, we detail the proposed MLRan dataset. In Section 5, we discuss the experimental design used to address the research questions, and in Sections Section 6, we present the research questions' results and compare them with related works, respectively. Finally, in Section 8, we conclude the paper and provide recommendations for future research.

2. Ransomware behavioural datasets and their limitations

This section presents a curated list of publicly available ransomware datasets suitable for training machine learning models and discusses their characteristics, contributions and limitations.

[1] VirusTotal Report on Ransomware in the Global Context: https://blog.virustotal.com/2021/10/ransomware-in-global-context.html

[2] MLRan GitHub Repository: https://github.com/faithfulco/mlran.

2.1. Ransomware datasets

The **EldeRAN** ransomware dataset[3] was published in 2016 (Sgandurra et al., 2016). It comprises 582 ransomware samples spanning 11 families, along with 942 samples of goodware applications. Using the Cuckoo Sandbox on Windows XP 32-bit, EldeRAN dynamically analyses both ransomware and goodware to extract a rich set of 30,967 features—including API calls, registry operations, file system and directory activities, file extensions, dropped files, and embedded strings. A feature selection algorithm then identifies the most relevant features used in a Regularised Logistic Regression classifier to distinguish between ransomware and goodware. The ransomware samples were randomly selected from VirusShare,[4] and the goodware samples were obtained from a software aggregator website,[5] ensuring a diverse set of benign applications.

Despite its widespread use, the Elderan dataset's sample size is relatively small and imbalanced, with 582 ransomware samples compared to 942 goodware samples. Moreover, the dataset predominantly comprises crypto-ransomware, lacking representation of newer ransomware types such as RaaS and modern multi-stage attacks. Consequently, it may not adequately reflect the current threat landscape, limiting its utility for developing robust, up-to-date detection models.

The **ShieldFS** dataset is a self-healing, ransomware-aware filesystem that intercepts low-level I/O Request Packets (IRPs) to detect and mitigate ransomware attacks in real time (Continella et al., 2016). The system was established by first developing IRPLogger, a custom kernel-level sniffer installed on volunteer machines to collect a comprehensive baseline of normal filesystem activity over one month—resulting in about 1.7 billion IRPs from 2245 different applications. Subsequently, known ransomware samples — 383 active instances from 5 families (CryptoWall, TeslaCrypt, Critroni, CryptoDefense, Crowti) — were executed on controlled machines configured with realistic decoy files and user data. By monitoring metrics such as file reads, writes, directory listings, and entropy changes, ShieldFS detects suspicious encryption-like behaviours and automatically rolls back malicious modifications. Although effective for mitigating file-encryption attacks, the dataset primarily captures file I/O patterns, lacks behavioural feature diversity (e.g., API calls and registry activity), and does not cover newer ransomware types such as RaaS and modern ransomware.

The **ISOT ransomware detection dataset**[6] was published in 2020 (Jethva et al., 2020). This dataset was designed to capture a diverse range of ransomware families and variants, comprising 666 ransomware samples alongside 103 goodware samples. Data was collected using the Cuckoo Sandbox in a controlled Windows 7 environment to simulate real-world conditions as closely as possible. The dataset includes 51,556 features grouped into eight categories: API calls, registry key operations, command line operations, DLLs, directory enumerations, mutexes, embedded strings, and miscellaneous binary attributes. Despite its strengths in feature diversity, the relatively small number of samples (769 samples) and imbalanced distribution of goodware samples, when compared to ransomware, may skew model training.

The **RanSAP** dataset[7] is an open dataset of ransomware storage access patterns designed for training machine learning models (Hirano et al., 2022). It captures time-series data from 7 prominent ransomware samples alongside 5 benign software samples. Using a thin hypervisor-based monitoring system, RanSAP records low-level storage interactions, such as sector-based read and write operations and entropy measurements, across various operating conditions. The dataset encompasses samples executed under different configurations, including operating system versions and storage device configurations (e.g., devices with full drive encryption enabled). By focusing on storage access patterns, RanSAP provides insight into the dynamic behaviours of ransomware that are critical for developing detection models. However, the dataset scope is confined to storage-related features, omitting other behavioural features such as API calls and registry activities that may also be valuable for comprehensive detection. Additionally, the relatively small sample size restricts its representativeness of the broader ransomware ecosystem, as the ransomware families considered are just crypto and RaaS, omitting locker and newer modern ransomware types.

The **Dynamic Feature Dataset**[8] is built from experiments on 40 artifacts (20 ransomware and 20 goodware) executed ten times on each of five victim devices, resulting in a total of 2000 experiments using the Cuckoo sandbox (Herrera-Silva and Hernández-Álvarez, 2023). The dataset comprises 50 distinct features organised into 7 feature groups, which capture dynamic behaviours such as API call flows, registry operations, file and directory activities, network communications, and other relevant runtime characteristics. Despite its balanced sample distribution and good experimental design, which leads to generating more observations, the dataset sample size is relatively modest compared to more extensive collections, which might restrict its representativeness across the full spectrum of ransomware types.

The **MarauderMap** ransomware dataset[9] comprises 7796 active Windows ransomware samples spanning 95 families (Hou et al., 2024). The samples were dynamically analysed using a custom testbed developed by the authors. During execution, the testbed collected 1.98TiB of runtime logs spanning six key behavioural categories: API calls, I/O operations, network traffic, registry modifications, command executions, and service management. Samples were sourced from platforms like VirusTotal and VX Vault, with the majority identified between 2021 and 2023. Despite its comprehensive scale of ransomware samples and rich behavioural features, the dataset does not include goodware samples. While the study used benign executables for evaluating detection strategies, these were not part of the dataset and were not analysed in the sandbox.

The **MIRAD** dataset,[10] introduced in 2024, is designed for dynamic ransomware detection (Marcinkowski et al., 2024). It was generated by simulating ransomware attacks in a virtual environment that mimics typical office activity on Windows. The dataset includes 13 features extracted from event logs (e.g., API calls, registry changes, file events) aggregated over fixed intervals with a moving average. Each data point is labelled as "pre-attack" (benign) or "post-attack" (ransomware activity), comprising a total of 67,427 samples (39,440 for training and 27,987 for testing), and includes 78 ransomware samples. However, the small number of ransomware samples and features may not capture the full spectrum of real-world behaviours, and the documentation lacks critical metadata, which could hinder reproducibility and further research.

The **Ransomware Combined Structural Feature (RCSF)** dataset[11] aggregates structural features from Windows PE files to support detection of emerging ransomware families through static analysis (Moreira et al., 2024). It combines multiple features, including PE header fields, imported DLLs, function calls, section entropy, and opcodes of each binary sample. The dataset comprises 2675 samples, with a training set of 1023 ransomware (from 25 families) and 1134 goodware, and a testing set of 385 ransomware (from 15 families) and 133 goodware. Feature extraction is performed using tools such as the Python pefile library, Detect It Easy, and Distorm3, followed by variance threshold feature selection to refine the data. An ensemble model combining

[3] Elderan: https://rissgroup.org/ransomware-dataset/.
[4] VirusShare: http://virusshare.com/.
[5] Software Informer: https://software.informer.com/software/.
[6] ISOT: https://onlineacademiccommunity.uvic.ca/isot/2022/11/27/botnet-and-ransomware-detection-datasets/.
[7] RanSAP: https://github.com/manabu-hirano/RanSAP/.
[8] Dynamic Feature Dataset: https://github.com/Juan-Herrera-Silva/Paper-SENSORS/tree/main.
[9] MarauderMap: https://github.com/m1-llie/MarauderMap-code.
[10] MIRAD: https://github.com/Sagenso/MIRAD.
[11] RCSF: https://data.mendeley.com/datasets/yzhcvn7sj5.

Table 1
Descriptive summary of the major publicly available ransomware datasets. N denotes the total number of samples, #Ran represents the number of ransomware samples, and #Good indicates the number of goodware samples. #Fx refers to the total number of features, while #RanF specifies the number of ransomware families. #FxG denotes the number of feature groups, and OS indicates the operating system used in the dataset. The symbol '-' indicates that the respective information was not found in the dataset/paper.

Dataset	Year	Analysis	N	#Ran	#Good	#Fx	#RanF	#FxG	Software Used	Sample Source	OS
Elderan	2016	Dynamic	1524	582	942	30 967	11	7	Cuckoo Sandbox	VirusShare Software informer	Win XP
ShieldFS	2016	Dynamic	2628	2245	383	6	5	2	IRPLogger, ShieldFS driver,	VirusTotal	Win 7
ISOT	2020	Dynamic	769	666	103	51 556	20	8	Cuckoo Sandbox	VirusTotal	Win 7
RanSAP	2022	Dynamic	12	7	5	5	7	1	BitVisor	ANY.RUN	Win Server 2008 R2 and Win 7
Dynamic Feature	2023	Dynamic	40	20	20	50	20	7	Cuckoo Sandbox	VirusShare, theZoo, VirusTotal and Hybrid-Analysis	5 Different Windows OS
MarauderMap	2024	Dynamic	7796	7796	–	–	95	6	Author Testbed	VirusTotal, Vx vault, hacker forums, dark web, black market groups	Win 10
MIRAD	2024	Dynamic	78	78	–	13	–	6	Author Testbed	MalwareBazaar	Win 10 & 11
RCSF	2024	Static	2675	1408	1267	11 027	25	4	pefile lib DIE Distorm3	VirusShare Hybrid-Analysis	Windows
RanSMAP	2025	Dynamic	12	6	6	23	6	2	BitVisor	ANY.RUN	Win 10
MLRan Our Dataset	2025	Dynamic	4880	2330	2550	6,467,926	64	9	Cuckoo Sandbox	VirusShare VirusTotal MalwareBazaar Hybrid-Analysis MarauderMap	Win 7

Logistic Regression, Random Forest, and XGBoost achieves over 97% in accuracy, precision, recall, and F-measure, with a prediction time of about 0.37 s per sample. While the data includes rich structural features, it relies exclusively on static features extracted from Windows PE files. If ransomware employs obfuscation, packing, or encryption techniques, the structural features may be altered, potentially reducing detection accuracy.

RanSMAP[12] captures low-level storage and memory access patterns using a thin hypervisor (Hirano and Kobayashi, 2025). Unlike its predecessor, RanSAP, which collected only storage access patterns, RanSMAP integrates memory access features to improve deep learning–based ransomware detection by about 2.3%, especially under simultaneous execution of ransomware and benign applications. The RanSMAP dataset comprises 1970 executions, which include 6 ransomware samples (WannaCry, Ryuk, REvil, LockBit, Darkside and Conti) and 6 goodware samples. Each execution is represented by a 23-dimensional feature vector, derived from two feature groups—5 storage access features and 18 memory access features. The dataset spans 6 distinct ransomware families and was collected on Windows 10. However, the dataset includes a relatively small number of ransomware samples and types, which may not fully represent real-world diversity across ransomware types.

Tables 1 and 2 provide, respectively, a descriptive summary and key characteristics of major publicly available ransomware datasets, highlighting their methodologies, sample composition, and feature representation. Most datasets rely on dynamic analysis, with RCSF being the only exception that employs static analysis. Commonly used tools include Cuckoo Sandbox, BitVisor, and IRP Logger, with Cuckoo Sandbox being the most widely used. Some datasets, such as MarauderMap and MIRAD, utilise proprietary testbeds, which limit reproducibility.

The sources of ransomware samples vary, with VirusTotal being the most frequently used repository. Additional datasets leverage sources such as MalwareBazaar, Hybrid-Analysis, and VirusShare, while MarauderMap uniquely obtains some samples from dark web markets and hacker forums, offering potentially more diverse data. In terms of operating systems, most datasets focus on Windows 7 and Windows 10, with only a few using multiple Windows versions in analysing the samples, such as the Dynamic Feature dataset. MLRan, however, integrates Cuckoo Sandbox and obtains ransomware samples from multiple malware repositories, providing a more comprehensive and diverse dataset for analysis.

[12] RanSMAP: https://github.com/manabu-hirano/RanSMAP.

2.2. Limitations

One of the most significant limitations of existing datasets is their small size. Only MarauderMap contains 3000 or more samples, while the rest remain relatively small. However, MarauderMap does not include goodware samples, making it unsuitable for training machine learning models that require both ransomware and benign software for accurate classification. Also, only 4 out of 9 datasets include a balanced representation of ransomware and goodware. MLRan addresses this limitation by offering a well-balanced dataset that comprises both ransomware and goodware, ensuring better training and generalisation for machine learning models.

Another key issue is the incomplete representation of ransomware types. No dataset comprehensively includes all ransomware types; datasets either prioritise recent ransomware at the expense of older samples or focus on past variants while neglecting modern threats. 8 out of 9 datasets include crypto ransomware, while locker and modern ransomware are significantly underrepresented. Additionally, 7 out of 9 datasets cover only up to 25 ransomware families, while MarauderMap includes 95 families, making it the most diverse. MLRan overcomes these gaps by providing full-spectrum coverage of historical and contemporary ransomware types across 64 families, ensuring better applicability in studying the evolution of ransomware attacks.

Goodware representation is another critical limitation. Although 7 out of 9 datasets include goodware, only 3 datasets have a diverse selection of benign software. This imbalance reduces the effectiveness of machine learning models, as a well-balanced dataset is essential for distinguishing between malicious and benign software. MLRan effectively resolves this issue by incorporating a diverse and balanced set of goodware, enhancing its real-world applicability.

Feature representation across these datasets also varies significantly. File and directory operations are included in all the datasets, reflecting ransomware's primary impact on file systems. However, API calls and registry keys are only included in 6 and 5 datasets, respectively, despite their importance in behavioural analysis. Additionally, network-based features, system processes, dropped operations, and signatures are rarely considered; that is, each feature is only included in two datasets, which limits the effectiveness of the datasets in detecting ransomware using dynamic behavioural indicators. MLRan incorporates all these critical features, ensuring a richer dataset that enhances machine learning-based ransomware detection.

MLRan addresses these shortcomings by offering a large, well-balanced dataset that includes diverse ransomware types, comprehensive goodware samples, and an extensive feature set, making it a superior dataset for advancing ransomware detection research.

Table 2
Key characteristics of major publicly available ransomware datasets. The presence of a specific characteristic is denoted by '✓' while its absence is marked as '×'. The symbol '-' indicates that the respective information was not found in the dataset. A dataset is considered large if it contains 3000 or more samples. Within ransomware types, L represents locker ransomware, C denotes crypto ransomware, Rs signifies ransomware-as-a-service, and M indicates modern ransomware. For goodware, G denotes the presence of goodware samples, while D indicates if the goodware samples are diverse. Regarding labelling, B represents binary labelling (ransomware vs. goodware), F denotes family-based labelling, and T signifies type-based labelling. Feature groups represent the different feature groups available in the datasets. The different groups include API calls (API), registry keys (REG), file operations (FILE), directory operations (DIR), string-based features (STR), network-related features (NET), system processes (SYS), dropped operations (DROP), and signatures (SIG). Any additional features not falling into these categories are classified as Others. The row, '% of ✓' indicates the percentage of datasets that have that characteristic, excluding our dataset, rounded to a whole number.

Dataset	Large Size	Ranamware Types				Goodware		Bal	Labelling			Feature Groups									
		L	C	Rs	M	G	D		B	F	T	API	REG	FILE	DIR	STR	NET	SYS	DROP	SIG	Others
EldeRAN	×	✓	✓	×	×	✓	✓	×	✓	✓	×	✓	✓	✓	✓	✓	×	×	✓	✓	×
SHIELDFS	×	×	✓	×	×	✓	✓	×	✓	✓	×	×	×	✓	✓	×	×	×	×	×	×
ISOT	×	✓	✓	×	✓	✓	×	×	✓	✓	×	✓	✓	✓	✓	✓	×	✓	✓	✓	✓
RanSAP	×	×	✓	✓	×	✓	×	✓	✓	✓	×	×	×	✓	✓	×	×	×	×	×	✓
Dynamic Feature	×	✓	✓	✓	×	✓	×	✓	✓	✓	×	✓	✓	✓	✓	×	✓	×	×	×	✓
MarauderMap	✓	×	✓	✓	✓	×	×	×	×	✓	×	✓	✓	✓	✓	✓	✓	✓	×	×	✓
MIRAD	×	–	–	–	–	×	×	×	✓	×	×	✓	✓	✓	✓	×	×	×	×	×	×
RCSF	×	×	✓	✓	✓	✓	✓	✓	✓	✓	×	✓	×	✓	✓	✓	×	×	×	×	✓
RanSMAP	×	×	✓	✓	×	✓	×	✓	✓	✓	×	×	×	✓	✓	×	×	×	×	×	✓
% of ✓	11	33	89	56	33	78	33	44	89	89	0	67	56	100	100	44	22	22	22	22	67
MLRan - Our Dataset	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

3. GUIDE-MLRan: Guidelines for developing high-quality ransomware datasets

Data quality is a critical determinant of machine learning effectiveness, encompassing both quantitative and qualitative aspects of data evaluation. Its definition varies based on context; for instance, Wang and Strong (1996) defines data quality as "fit for use", while (Wand and Wang, 1996) describes it as the alignment between "real-world and system states". More recently, Zhou et al. (2024) defined high-quality data as meeting user needs and being purpose-fit for ML tasks, a definition that we adopt in this study. Furthermore, Gong et al. (2023) emphasises that a high-quality dataset should accurately represent real-world phenomena and be comprehensive and free from biases.

Poor data quality has a profound impact on ML systems. Issues such as mislabelling, inaccuracy, inconsistency, duplication, and overlap can severely impair even the most advanced ML models (Chen et al., 2021; Zhou et al., 2024; Schwabe et al., 2024). In the cybersecurity domain, Tran et al. (2022) empirically demonstrated how data quality impacts ML performance for cyber intrusion detection by evaluating eleven datasets. Their findings underscore the significant influence of data quality on both conventional machine learning models and pre-trained language models, reinforcing the need for rigorous dataset development practices.

While there have been efforts to establish evaluation guidelines for ransomware mitigation solutions, dataset construction has received comparatively little attention. McIntosh et al. (2021) introduced unified metrics for evaluating ransomware mitigation studies, categorising them into evaluation, output, versatility, and strength. However, their framework does not address how ransomware datasets should be developed, leaving a critical gap in the field. Additionally, while (Zhou et al., 2024) identified eight key data quality dimensions — including completeness, consistency, timeliness, confidentiality, accuracy, standardisation, unbiasedness, and ease of use — these principles were formulated in the context of general ML datasets, with no specific focus on security, malware, or ransomware.

Related work in cybersecurity dataset development has primarily focused on intrusion detection systems (IDSs). Shiravi et al. (2012) proposed a systematic methodology for generating benchmark IDS datasets, advocating for realistic network traffic capture, accurate labelling, complete interaction records, and comprehensive packet content preservation. Their approach ensures that datasets closely reflect real-world attack conditions, thus improving the reliability of IDS evaluations. Similarly, Tran et al. (2022) proposed nine criteria — reputation, relevance, comprehensiveness, timeliness, variety, accuracy, consistency, duplication, and overlap — to ensure the high quality of IDS datasets.

Although some of these IDS dataset guidelines can be applied to ransomware datasets, they do not account for the unique characteristics of ransomware attacks and the specific requirements of ML-based ransomware detection models. Ransomware exhibits distinct attack behaviours, such as file encryption and system modification, which require a tailored approach to dataset development.

To address this gap, we propose GUIDE-MLRan, a set of guidelines inspired by previous work for developing high-quality behavioural ransomware datasets for ML-based ransomware detection. GUIDE-MLRan is summarised in Table 4. The guidelines are based on five key principles:

1. *Sample diversity and representativeness*, ensuring a balanced inclusion of diverse ransomware families and goodware;
2. *Sample quality and accuracy*, emphasising correct labelling, sufficient dataset size, and samples that cover an extended period;
3. *Sandbox and testbed requirements*, advocating for realistic execution environments;
4. *Representative feature extraction*, capturing comprehensive, standardised behavioural features; and
5. *Documentation, reproducibility, and data extension*, promoting metadata completeness, ethical considerations, public availability, and continuous dataset updates.

These principles can support the development of robust, reproducible, and well-structured datasets for advancing ransomware detection research. Although these guidelines has been developed specifically for ransomware datasets, they can be applicable to malware datasets in general.

3.1. Sample diversity and representativeness

Sample diversity, defined as the inclusion of varied samples, and representativeness, which ensures alignment with real-world distributions, are critical for machine learning datasets (Chio and Freeman, 2018). Bengio et al. (2017) highlight that diverse samples enhance model generalisation, enabling them to handle rare and complex scenarios effectively. Similarly, Hastie et al. (2017) argues that representativeness improves generalisability by ensuring datasets reflect real-world distributions. However, most publicly available ransomware datasets fail to meet these criteria, as collecting diverse and representative samples of ransomware and goodware remains a significant challenge.

A significant issue in constructing ransomware datasets is sample selection bias, where the collected data does not adequately reflect the true distribution of the underlying security problem, which limits model effectiveness (Arp et al., 2022; Cortes et al., 2008). In

cybersecurity applications, sampling from the true distribution is inherently difficult and, in some cases, infeasible. While this bias cannot be entirely eliminated, it can be mitigated by ensuring that goodware and ransomware samples are diverse and balanced.

3.1.1. Diverse ransomware samples

Ransomware has undergone significant evolution, adapting to technological advancements and security measures over time. Initially, from locker ransomware, which restricts device access until a ransom is paid (e.g., Reveton, Kovter), to crypto-ransomware, which encrypts files and demands payment for decryption, often causing irreversible data loss (e.g., WannaCry, CryptoLocker) (Sgandurra et al., 2016). The rise of ransomware-as-a-service further expanded its reach by providing ready-made ransomware kits, enabling even non-expert attackers to launch sophisticated attacks (e.g., Hive, GandCrab) (Fisher et al., 2025). Modern ransomware integrates RaaS with advanced evasion techniques, such as polymorphism and metamorphism, to bypass traditional detection mechanisms (e.g., Conti, LockBit) (Raj et al., 2024). Furthermore, modern attacks often include double and triple extortion. Double extortion encrypts data and steals sensitive information, threatening to leak it if the ransom is not paid. Triple extortion adds pressure by targeting third parties or launching DDoS attacks, increasing disruption (Kerns et al., 2022; Anand and Shanker, 2023).

Capturing this evolution in datasets ensures comprehensive coverage of ransomware behaviours, enhancing machine learning models' ability to detect, classify, and respond to both known and emerging threats (Bengio et al., 2017; Hastie et al., 2017; Arp et al., 2022; Cortes et al., 2008; Miranda et al., 2022). However, obtaining ransomware samples can be challenging and downloading them individually can be time-consuming.

3.1.2. Diverse goodware samples

By exposing machine learning models to a broad set of benign applications, diverse goodware samples enhance generalisation, enabling models to distinguish legitimate software from ransomware across various computing environments. Moreover, they prevent bias by ensuring that models do not disproportionately classify certain software categories as malicious (Ashraf et al., 2019). Additionally, exposure to a wide range of goodware refines detection boundaries, improving the ability to accurately classify known and previously unseen applications (Sgandurra et al., 2016; Homayoun et al., 2017; Miranda et al., 2022).

To achieve comprehensive goodware representation, Botacin et al. (2021) underscores the importance of curating software that reflects real-world environments, particularly those commonly installed by users within the target system. A well-diversified selection should include applications from multiple categories, such as productivity tools (e.g., Microsoft Office), communication platforms (e.g., Zoom), internet tools (e.g., Firefox), system utilities (e.g., UltraViewer), antivirus software (e.g., Kaspersky Internet Security), lifestyle applications (e.g., Memory-Map), games (e.g., City Car Driving), developer tools (e.g., Python), business software (e.g., Money Manager), and educational programs (e.g., Dictionary). This variety ensures that machine learning models capture normal software behaviours across different domains, reducing the likelihood of misclassification (Herrera-Silva and Hernández-Álvarez, 2023).

Furthermore, goodware samples should be sourced from different time periods, incorporating both historical and contemporary software to account for evolving behavioural patterns. This temporal diversity is essential for building realistic datasets, capturing changes in software functionality and execution patterns, and ensuring models remain effective against evolving threats. By encompassing a broad and representative set of goodware, ransomware detection datasets can be used as accurate benchmarks to evaluate the performance and robustness of machine learning models, leading to more reliable and adaptable cybersecurity solutions (Miranda et al., 2022).

However, obtaining diverse goodware samples is challenging due to the vast software landscape, licensing restrictions, and the need for diverse representation. Manually curating such a dataset is time-consuming and requires careful selection to avoid bias while ensuring relevance to real-world environments.

3.1.3. Balanced class distribution

A balanced dataset in machine learning ensures that each class is represented in approximately equal proportions, which prevents model bias and enhances predictive performance. Class imbalance, on the other hand, occurs when one class significantly outnumbers another, leading to biased learning and poor generalisation (Miranda et al., 2022).

In ransomware detection, maintaining a balanced ratio of ransomware to goodware samples is crucial to prevent the model from favouring one class over the other (Onwuegbuche et al., 2023; Bengio et al., 2017). However, class imbalance in ransomware datasets exists at two levels: inter-class imbalance (between ransomware and goodware) and intra-class imbalance (within ransomware families) (He and Garcia, 2009). While achieving inter-class balance is feasible, particularly in dynamic analysis, maintaining intra-class balance is significantly more challenging. This difficulty arises from the natural imbalance in ransomware family prevalence, the rapid evolution of ransomware types, and the scarcity of certain ransomware families (Rahman et al., 2022; Botacin et al., 2021). Some families are widely distributed and frequently analysed, while others remain rare and underrepresented (Gong et al., 2023).

Despite these challenges, ensuring diversity within ransomware datasets remains essential. A well-balanced dataset should incorporate varied ransomware types and families, reflecting real-world threats while mitigating bias. Achieving a reasonable distribution of ransomware types and families enhances model robustness, enabling machine learning systems to generalise better and improve ransomware detection across different variants.

3.2. Sample quality and accuracy

Ensuring sample quality and accuracy is crucial for developing effective machine learning models, particularly in the context of ransomware detection. This requires accurate sample labelling, representativeness and covered time period.

3.2.1. Accurate labelling

Accurate labelling is fundamental for supervised learning, ensuring that models are trained on reliable and meaningful data. Gong et al. (2023) define accuracy as the degree to which data attributes correctly represent the true values of a given concept. In contrast, Cai and Zhu (2015) highlight that accuracy relies on alignment with an agreed "source of truth". In ransomware detection, labelling accuracy directly impacts detection performance, as incorrect labels can introduce significant bias into the learning process, making systems vulnerable to poisoning attacks and poor detection accuracy (Arp et al., 2022; Apruzzese et al., 2022; Marcinkowski et al., 2024).

Labelling ransomware samples is particularly challenging due to the lack of readily available ground truth data, creating a chicken-and-egg problem where detection relies on accurate labels, yet obtaining those labels is non-trivial (Ceschin et al., 2024; Arp et al., 2022). The rapid evolution of malware further complicates this process, often requiring manual verification. Manual expert analysis is the most reliable method, but is time-consuming and impractical for large datasets (Verma et al., 2019). To address this challenge, researchers sometimes use VirusTotal Antivirus (AV) majority voting, accepting only malware samples with consistent labels across multiple AV vendors. However, this reduces dataset size and may introduce selection bias, limiting generalisation to ambiguous threats (Ceschin et al., 2024).

Efforts to standardise malware classification labels are ongoing, with automated labelling tools playing a crucial role. Tools such as Euphony for Android malware (Hurier et al., 2017) and AVClass for Windows malware (Sebastián et al., 2016) help streamline classification. A well-labelled dataset should also include detailed metadata, such as ransomware family, type, and sample hashes, enabling nuanced analysis beyond binary classification and reproducibility of results.

One of the common pitfalls in malware research is assuming that all crawled applications from legitimate websites are benign without validation, which can compromise dataset integrity (Botacin et al., 2021). To mitigate this problem, researchers should rigorously verify benign samples, for instance, by submitting file hashes to VirusTotal and selecting only those with zero detections across all antivirus engines (Pendlebury et al., 2019). Ensuring high-quality, well-verified labels is crucial for improving model reliability, reducing false positives, and strengthening ransomware detection systems.

3.2.2. Sample size

Determining the sufficient number of ransomware and goodware in a dataset to ensure their representativeness is a complex question with no definitive answer. It depends on factors, such as research objectives, the availability of ransomware samples in the wild, the machine learning techniques used, and the level of generalisation required (Hirano et al., 2022; Botacin et al., 2021; Apruzzese et al., 2022). While some methods, such as deep learning, benefit from larger datasets (Bengio et al., 2017), increasing data volume beyond a certain threshold does not always improve performance. Instead, optimising feature selection and representation strategies may yield better results (Ceschin et al., 2024). However, representativeness is more critical than size. A small but well-curated dataset that accurately reflects real-world ransomware behaviours is more valuable than a large, biased, or unrepresentative dataset (Ceschin et al., 2024). Moreover, behavioural ransomware datasets tend to be smaller than static datasets, as behavioural analysis requires individual sample execution and manual validation, making data collection and analysis time-consuming. In summary, the dataset should be large enough to capture common ransomware behaviour patterns while maintaining high-quality, representative samples to enhance model reliability.

3.2.3. Covered time period

The time period a dataset covers refers to its suitability for a given task based on its age and longevity (Gong et al., 2023; Tran et al., 2022). Yaseen (2023) found that models trained exclusively on new ransomware struggle to detect older variants, and vice versa, due to evolving feature sets. However, models trained on temporally diverse datasets achieved very high accuracy, demonstrating the importance of incorporating both old and new samples to enhance generalisation and detection robustness. Since ransomware actors often revive outdated attack methods, it is also crucial to retain historical ransomware samples, such as locker ransomware, alongside modern variants. Similarly, including both recent and older goodware versions improves model generalisation. To maintain temporal consistency and track ransomware evolution, datasets should include timestamps for collected samples (Pendlebury et al., 2019).

3.3. Sandbox hardening and comprehensive behavioural analysis

A sandbox in ransomware analysis is a controlled virtual environment used to execute and observe ransomware behaviour safely without risking harm to the host system or network (Sgandurra et al., 2016; Ahmed, 2024).

3.3.1. Sandbox hardening

Sandbox hardening in ransomware behavioural analysis involves strengthening the sandbox environment to resist detection and evasion techniques employed by ransomware (Alrawi et al., 2024). The objective is to create an analysis environment that closely resembles a real user system, ensuring ransomware executes its full range of malicious behaviours without detecting the sandbox (Leguesse et al., 2018). Without effective hardening, many ransomware samples alter their behaviour, delay execution, or terminate when they detect they are running in a sandbox, leading to incomplete or misleading results.

Some malware employs evasion techniques to detect sandbox environments. MITRE ATT&CK[13] categorises these into three main types:

1. System-based checks – Ransomware inspects system attributes to determine if it is running in a sandbox. It checks CPU core count, installed programs, memory, disk size, digital signatures, and system artefacts. If these do not match a real user environment, the malware may refuse to execute.
2. User activity-based checks – Since sandboxes lack human interactions, ransomware may remain dormant until it detects user activity such as mouse movements, document scrolling, opening applications, or browsing history. Without these actions, the malware assumes it is in a sandbox and delays execution.
3. Time-based evasion – Sandboxes typically execute malware for a limited time, making them vulnerable to delayed execution techniques. Ransomware exploits this by using extended sleep delays, logic bombs, or stalling code, ensuring execution occurs only after the sandbox analysis period expires

Beyond these methods, encryption, obfuscation, and dynamic system alterations further complicate malware detection.[14] Botacin et al. (2021) highlight the difficulty of ensuring successful execution in sandbox environments, as some samples fail due to corruption, OS incompatibilities, or evasion tactics. To mitigate this challenge, defining clear execution criteria, such as a minimum number of API calls or observable behaviours, and reporting execution success rates, improves dataset reliability.

To counter sandbox-aware ransomware, hardening strategies should simulate user activity (e.g., opening files, internet browsing), mimic system configurations (e.g., CPU, RAM, OS settings), randomise execution times, and emulate hardware characteristics. These measures reduce detection risk and encourage ransomware to execute fully, capturing more accurate behavioural data.

Execution timeouts are also critical. Küchler et al. (2021) found that 98% of executed basic blocks occur within the first two minutes, making a 120-second timeout optimal for capturing malicious behaviour without excessive resource use. This result supports (Willems et al., 2007), confirming that a two-minute threshold is sufficient for analysing freshly collected malware samples in most cases. Therefore, we implemented a 2-minute (120-second) timeout in this research. For more sandbox-specific information, we refer the reader to Alrawi et al. (2024).

3.3.2. Comprehensive behavioural analysis

Gong et al. (2023) state that a sandbox should collect a comprehensive set of attributes for ransomware behavioural analysis. These should include application programming interface (API) calls, registry modifications (REG), file operations (FILE), directory activities (DIR), embedded strings (STR), network traffic (NET), system processes (SYS), dropped files (DROP), and signatures (SIG). Missing these attributes can lead to incomplete ransomware profiling, limiting feature extraction for machine learning and detection accuracy. Therefore, collecting these attributes during dynamic analysis and generating a coherent report is crucial for accurate classification (Or-Meir et al., 2019).

[13] MITRE ATT&CK - Virtualization/Sandbox Evasion: https://attack.mitre.org/techniques/T1497/.

[14] Apriorit: https://www.apriorit.com/dev-blog/545-sandbox-evading-malware.

3.4. Representative feature extraction and modelling

Representative feature extraction ensures that datasets include the most informative attributes, enhancing model accuracy and interoperability.

3.4.1. Relevant feature extraction

After thoroughly analysing ransomware samples and generating a structured report, the next crucial step is extracting comprehensive and relevant features for machine learning-based detection.

Kelleher et al. (2020) emphasises that selecting meaningful features enhances model performance, efficiency, and interpretability. In ransomware detection, this involves extracting critical behavioural feature groups, such as API calls and registry keys, as outlined in C8 (see Section 3.3.2), to ensure a detailed and representative characterisation of sample activity.

To automate feature extraction, we develop Python scripts[15] that parse Cuckoo Sandbox reports, extracting the nine key behavioural feature groups discussed in C8 (see Section 3.3.2). These scripts are available publicly to promote reproducibility and standardisation, enabling researchers to streamline feature extraction and enhance ransomware detection frameworks.

3.4.2. Data preprocessing

The phrase "garbage in, garbage out" highlights the importance of data preprocessing in machine learning, ensuring that ransomware detection models are trained on clean and well-formatted data (Kang and Tian, 2018).

A crucial step in preprocessing is data cleaning, which involves removing duplicates, handling missing values, and standardising feature distributions. Duplicate instances distort evaluation metrics, leading to misleading performance results (Chen et al., 2021; Zhou et al., 2024). Similarly, excessive missing values introduce bias, reducing model reliability (Cai and Zhu, 2015). Normalisation and standardisation further ensure consistent feature scales, preventing specific attributes from disproportionately influencing the model.

Beyond cleaning, the dataset format must align with the intended machine learning approach. Sequential data formats (e.g., time-series logs) suit recurrent neural networks (RNNs) for analysing temporal ransomware behaviour, while tabular formats (e.g., CSV, databases) are better suited for classification algorithms like decision trees and logistic regression. Graph-based structures can also help identify ransomware propagation patterns (Andronio et al., 2015).

Feature selection refines the dataset by identifying the most relevant and discriminative attributes, improving model efficiency, interpretability, and generalisation. Approaches include filter methods (statistical relevance), wrapper methods (iterative subset evaluation), and embedded methods (selection during model training). A multi-stage hybrid approach combining these techniques helps eliminate redundancy while retaining essential behavioural indicators (Onwuegbuche et al., 2023).

3.4.3. Model training and evaluation

Model training requires careful design choices, bias prevention, and hyperparameter tuning to ensure robust generalisation. A critical issue during training is data snooping bias, which occurs when information from the test set is inadvertently introduced into the training process, leading to overoptimistic performance estimates and poor real-world applicability. Arp et al. (2022) identifies three major types of data snooping bias that must be avoided.

1. *Test snooping* occurs when test data is misused for feature selection or hyperparameter tuning, leading to overfitting and misleading performance metrics. To prevent this, training, validation, and test sets must remain strictly separate, with test data used only for final evaluation.
2. *Temporal snooping* happens when past and future data are mixed during training, ignoring real-world time dependencies. This causes models to perform well on historical data but fail on new ransomware strains. To avoid this, dataset splits should preserve temporal order to reflect ransomware evolution.
3. *Selective snooping* involves preprocessing based on future knowledge, such as removing outliers using the entire dataset. This creates bias, as such insights would not be available at the time of deployment. Instead, preprocessing should rely only on information accessible during training to ensure fairness and realism.

Beyond avoiding biases, models should be trained on diverse and well-balanced datasets that accurately represent real-world ransomware behaviours. Relying on imbalanced datasets can lead to models that favour the majority class, reducing effectiveness in detecting novel ransomware strains. Additionally, hyperparameter tuning using techniques such as cross-validation helps improve model generalisation while mitigating overfitting.

A robust evaluation framework is essential to assess how well a model generalises to real-world ransomware threats. As Botacin et al. (2021) suggest, evaluation criteria should be clearly defined from the start to align with deployment requirements. The evaluation dataset must be representative of the operational environment, avoiding temporal and spatial biases. Pendlebury et al. (2019) highlight that model performance can degrade when tested on data from different time periods or distributions than those used in training. Furthermore, the presence of duplicate samples in the test set can inflate performance metrics, leading to an inaccurate assessment of model effectiveness (Tran et al., 2022).

Choosing the right evaluation metrics is crucial for obtaining a comprehensive performance assessment. While accuracy is often reported, it is insufficient for ransomware detection due to the severe consequences of undetected threats. Instead, evaluation should incorporate Precision, Recall, F1 Score, and False Positive Rate (FPR) to measure detection effectiveness comprehensively. Given the risks associated with undetected ransomware, False Negative Rate (FNR) is particularly important, as a high FNR indicates a failure to detect active ransomware infections (Zhou et al., 2023). Additionally, class imbalance is common in ransomware datasets, meaning that balanced accuracy and the F1 score should be prioritised over raw accuracy (Eren et al., 2023). When evaluating models over time, particularly in continual learning settings, performance robustness should be assessed using metrics such as Area Under Time (AUT) to account for concept drift (Pendlebury et al., 2019).

To ensure meaningful comparisons, models should be benchmarked against strong baselines. Using overly complex models without evaluating their performance relative to simpler, well-established approaches can result in unjustified computational overhead. Automated Machine Learning (AutoML) frameworks can assist in identifying competitive baselines, ensuring that performance gains are due to meaningful improvements rather than excessive model complexity (Arp et al., 2022).

Additionally, Rossow et al. (2012) emphasises analysing false positives and false negatives to uncover system limitations and improve detection robustness, as error rates alone offer limited insight. In our work, we perform a thorough misclassification analysis using explainable AI methods.

3.5. Documentation, reproducibility, and data extension

Comprehensive documentation, reproducibility, and regular data updates are essential for creating reliable ransomware datasets. De-

[15] MLRan Cuckoo JSON Report Parsers: https://github.com/faithfulco/mlran/tree/main/4_cuckoo_parser_scripts.

tailed metadata and transparent processes ensure that datasets are understandable, reusable, and adaptable. These practices enhance collaboration, validate results, and maintain relevance in addressing evolving ransomware threats. Botacin et al. (2021) highlights that using non-reproducible methodologies for dataset definition and experimental design is a significant pitfall in malware research. Reproducibility is essential for scientific integrity as it enables other researchers to verify or challenge results, thereby advancing the field.

3.5.1. Availability of contextual and metadata information

Providing comprehensive contextual and metadata information about samples, sandboxes, datasets, and methods is fundamental to ensuring transparency, interpretability, and reproducibility in ransomware detection research (Gong et al., 2023; Rossow et al., 2012). Without detailed documentation, experimental findings become difficult to validate, compare, or replicate, hindering progress and limiting the applicability of research (McIntosh et al., 2021; Holzinger et al., 2019).

Metadata at the sample level must capture essential attributes such as sample type, source, hash, description, timestamp, and classification (ransomware family and type or goodware category). These details provide critical context for assessing dataset representativeness, preventing biases such as temporal snooping, and ensuring robust experimental design (Cai and Zhu, 2015). For instance, to avoid temporal snooping, it is critical for the timestamp of each sample to be provided (Arp et al., 2022).

At the dataset level, documentation should include collection methods, preprocessing steps, feature extraction techniques, and class distribution. This information is crucial for reconstructing datasets, adapting them to new research objectives, and ensuring fair comparisons between detection models (Herrera-Silva and Hernández-Álvarez, 2023). Without clear documentation, differences in data preprocessing — such as handling of missing values or selection of dynamic behavioural features — can lead to inconsistent model performance and misleading conclusions.

Equally important is documenting the sandbox or testbed settings used for sample analysis. Sandboxes differ in their configurations, execution parameters, and evasion countermeasures, which can significantly impact how ransomware behaves during execution. Therefore, system details such as OS version, installed software, hardware specifications, sandbox type (e.g., Cuckoo, BitVisor), execution timeouts, network connectivity settings, and anti-evasion mechanisms must be explicitly recorded (Rossow et al., 2012). Inadequate sandbox documentation compromises result reproducibility and makes it difficult to assess detection effectiveness across different environments (Alrawi et al., 2024; Leguesse et al., 2018).

Beyond the dataset and sandbox documentation, methodological transparency is essential. Reporting hyperparameter settings, feature selection strategies, training–validation splits, and evaluation metrics ensures that model comparisons are fair and reproducible. Cai and Zhu (2015) highlight that poorly documented methods introduce ambiguity, making it difficult to distinguish whether performance variations arise from the model's capability or differences in data handling.

3.5.2. Public availability of data and code

Open access allows independent validation, facilitates model comparisons, and drives innovation (Praveen and Almobaideen, 2023). High-quality datasets should be hosted on stable repositories (e.g., Zenodo, IEEE DataPort) with version control and open-access licensing (e.g., Creative Commons or GNU GPL) to ensure long-term availability and broad adoption (Zhou et al., 2024). Likewise, publishing code for data processing, model training, and evaluation prevents inconsistencies in feature extraction and preprocessing, ensuring results are replicable (Arp et al., 2022). Publicly accessible code allows other researchers to validate results, refine methodologies, and build upon existing work, contributing to the collective advancement of the field.

3.5.3. Ethical and legal considerations

Ensuring ethical and legal compliance when curating and sharing ransomware datasets is essential to prevent misuse, protect privacy, and adhere to regulatory standards. Researchers must balance open data access with responsible handling to maintain transparency while mitigating risks associated with malware distribution and sensitive data exposure (Thomas et al., 2017). Datasets should comply with cybersecurity laws, institutional guidelines, and ethical frameworks, such as the General Data Protection Regulation (GDPR) in the EU and the Computer Fraud and Abuse Act (CFAA) in the USA. Adhering to third-party data usage policies and licensing agreements, such as those from VirusTotal, is also critical to ensure regulatory compliance (Nasir et al., 2024).

To prevent misuse, ransomware source code should not be publicly released. Instead, researchers should share derived features, metadata, and execution logs, which provide meaningful insights while minimising security risks (Thomas et al., 2017). If access to ransomware binaries is necessary, controlled distribution mechanisms — such as request-based access for vetted researchers — should be implemented to prevent unauthorised use. Additionally, datasets must be anonymised to remove personally identifiable information (PII), including usernames, file paths, IP addresses, and domain names, to prevent privacy breaches.

Ensuring security and containment when handling live ransomware samples is essential to prevent unintended harm. Ransomware analysis systems must be fully isolated from real systems and networks, following institutional security policies (Alrawi et al., 2024). Implementing containment measures helps redirect malicious traffic and mitigate risks, making it a key ethical consideration. Additionally, any security breaches within the containment environment should be monitored and documented to improve system resilience and enhance future threat mitigation strategies (Rossow et al., 2012).

Ethical considerations also require clear documentation outlining the dataset's intended use, risks, and restrictions. Responsible disclosure ensures that datasets support cybersecurity research without enabling malicious activities. Researchers should consult ethical review boards or cybersecurity committees when handling datasets that pose dual-use risks, meaning they could be exploited for harmful purposes (Gao et al., 2024). Moreover, appropriate licensing (e.g., Creative Commons or GNU GPL) should define terms of use, ensuring responsible data sharing while acknowledging contributors.

By following these guidelines, researchers can curate and share ransomware datasets responsibly, promoting compliance, security, and collaboration while ensuring that datasets remain valuable for advancing ransomware detection research.

3.5.4. Continuous updates

Given the rapid evolution of ransomware, datasets must be continuously updated to capture new variants and emerging behavioural patterns. Regular updates enhance dataset relevance, ensuring machine learning models remain effective against evolving threats (Herrera-Silva and Hernández-Álvarez, 2023). By openly releasing our dataset and code, we enable researchers to extend, refine, and adapt the dataset, fostering collaborative advancements in ransomware detection as attack strategies evolve.

4. The MLRan dataset

This section describes the MLRan dataset, the methods used for sample collection and analysis, and the feature extraction techniques we employed to run the ML experiments. The compliance of the MLRan dataset with the GUIDE-MLRan guidelines is summarised in Table 4 and further elaborated in this section.

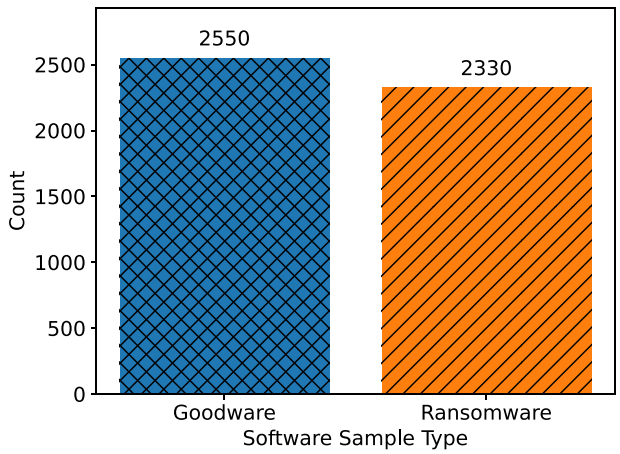


Fig. 1. Distribution of software sample types in the MLRan dataset. The dataset contains a total of 4880 samples, split into 2550 (52.25%) Goodware and 2330 (47.75%) Ransomware. The dataset is relatively balanced, with only a slight difference between the two categories.

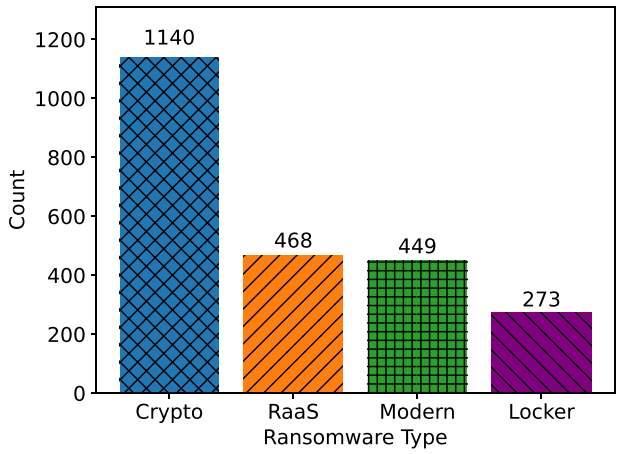


Fig. 2. Distribution of ransomware types in the MLRan dataset. The dataset contains a total of 2330 Ransomware samples, split into 1140 (48.92%) Crypto, 468 (20.08%) RaaS, 449 (19.27%) Modern, and 273 (11.72%) Locker.

4.1. Dataset description

Fig. 1 illustrates the distribution of software sample types within the MLRan dataset. The dataset contains a total of 4880 samples, with 2550 (52.25%) representing Goodware and 2330 (47.75%) corresponding to Ransomware. This shows a relatively even distribution between the two categories, with only a small, negligible proportion difference. Such a balanced dataset is advantageous for training machine learning models, as it reduces the risk of class imbalance affecting model performance. This balance ensures that the model can effectively differentiate between Goodware and Ransomware without being biased towards one class.

Fig. 2 shows the distribution of ransomware types in the MLRan dataset. Out of a total of 2330 ransomware samples, 48.92% (1140 samples) belong to the Crypto type, making it the most prevalent type in the dataset. The next most common types are RaaS with 20.08% (468 samples), followed by Modern at 19.27% (449 samples), and finally, Locker, which accounts for 11.72% (273 samples). A similar distribution pattern is observed at the family level, as illustrated in Fig. 3. The dataset contains a total of 64 ransomware families. Specifically, 32 families belong to the Crypto type, 15 are categorised as Modern, 13 fall under RaaS, and 4 correspond to the Locker type. This distribution satisfies the criteria (C1) of including diverse ransomware samples.

The distributions of ransomware types in Fig. 2 and ransomware families in Fig. 3 in the MLRan dataset reflect ransomware's current and historic distribution in the wild. While locker ransomware, which locks users out of their systems without encrypting files, still poses a threat, it is currently less prevalent than other types. Historically, crypto ransomware has been the most prominent, with several high-profile families making headlines. For instance, according to the UK's (National Cyber Security Centre, 2018) report, the WannaCry attack in 2017 infected over 300,000 computers across 150 countries, including the UK's National Health Service (NHS).

Crypto ransomware remains a significant threat. Emerging insights from the VirusTotal 2021 report[16] reveal that 95% of all ransomware samples were Windows-based executable files or dynamic link libraries, typically associated with this type. However, the landscape has shifted towards RaaS and more sophisticated modern ransomware variants. RaaS has seen a dramatic rise in recent years, contributing significantly to the increase in ransomware attacks. Notable RaaS families include Ryuk, which was responsible for one-third of all ransomware attacks in 2020, and Cerber, one of the earliest RaaS offerings.[17]

Modern ransomware attacks often combine multiple techniques and have become increasingly sophisticated. According to Statista,[18] prominent families such as LockBit, Akira, BlackCat, Play, and Royal are the top five most-detected ransomware attacks in 2023, which are all included in our dataset, as illustrated in Fig. 3. These modern variants often incorporate features from crypto and RaaS ransomware types while adding new tactics such as double or triple extortion, which involves not only encrypting data but also threatening to leak sensitive information obtained through data exfiltration or launch DDoS attacks.[19]

Furthermore, the MLRan dataset encompasses various goodware samples, as illustrated in Fig. 4. These samples span 11 broad categories of legitimate applications, comprising a total of 2550 samples. The most commonly used applications by individuals constitute the largest proportion, accounting for 47.52% (1212 samples). In addition, the dataset includes applications from 10 other categories, ensuring a comprehensive representation of goodware. Specifically, productivity applications account for 7.33% (187 samples), followed by developer tools at 6.71% (171 samples), communications software at 6.67% (170 samples), and lifestyle applications at 6.20% (158 samples). Moreover, business-related applications represent 5.88% (150 samples), while antivirus and security software comprise 5.02% (128 samples). Other categories include internet tools (4.04%, 103 samples), system tools (3.33%, 85 samples), and games, which form the smallest category at 1.65% (42 samples).

The inclusion of these diverse, balanced goodware and ransomware samples ensures that MLRan satisfies the guidelines for sample diversity and representativeness (C1, C2 and C3).

To demonstrate that the dataset meets the timeliness criterion (C6), we present a stacked bar chart in Fig. 5, illustrating the distribution of ransomware and goodware samples across different years. The plot reveals that ransomware samples in the MLRan dataset span from 2008 to 2024, while goodware samples cover a slightly broader period from 2006 to 2024.

Between 2006 and 2011, the dataset predominantly comprises goodware samples, reflecting the earlier prevalence of benign software.

[16] VirusTotal report on Ransomware in the Global Context: https://blog.virustotal.com/2021/10/ransomware-in-global-context.html.

[17] Security Magazine: https://www.securitymagazine.com/articles/93769-ryuk-ransomware-responsible-for-one-third-of-all-ransomware-attacks-in-2020.

[18] Statista: https://www.statista.com/statistics/1475291/most-detected-ransomware-types-worldwide/.

[19] Sealpath: https://www.sealpath.com/blog/ransomware-raas-operations-guide/.

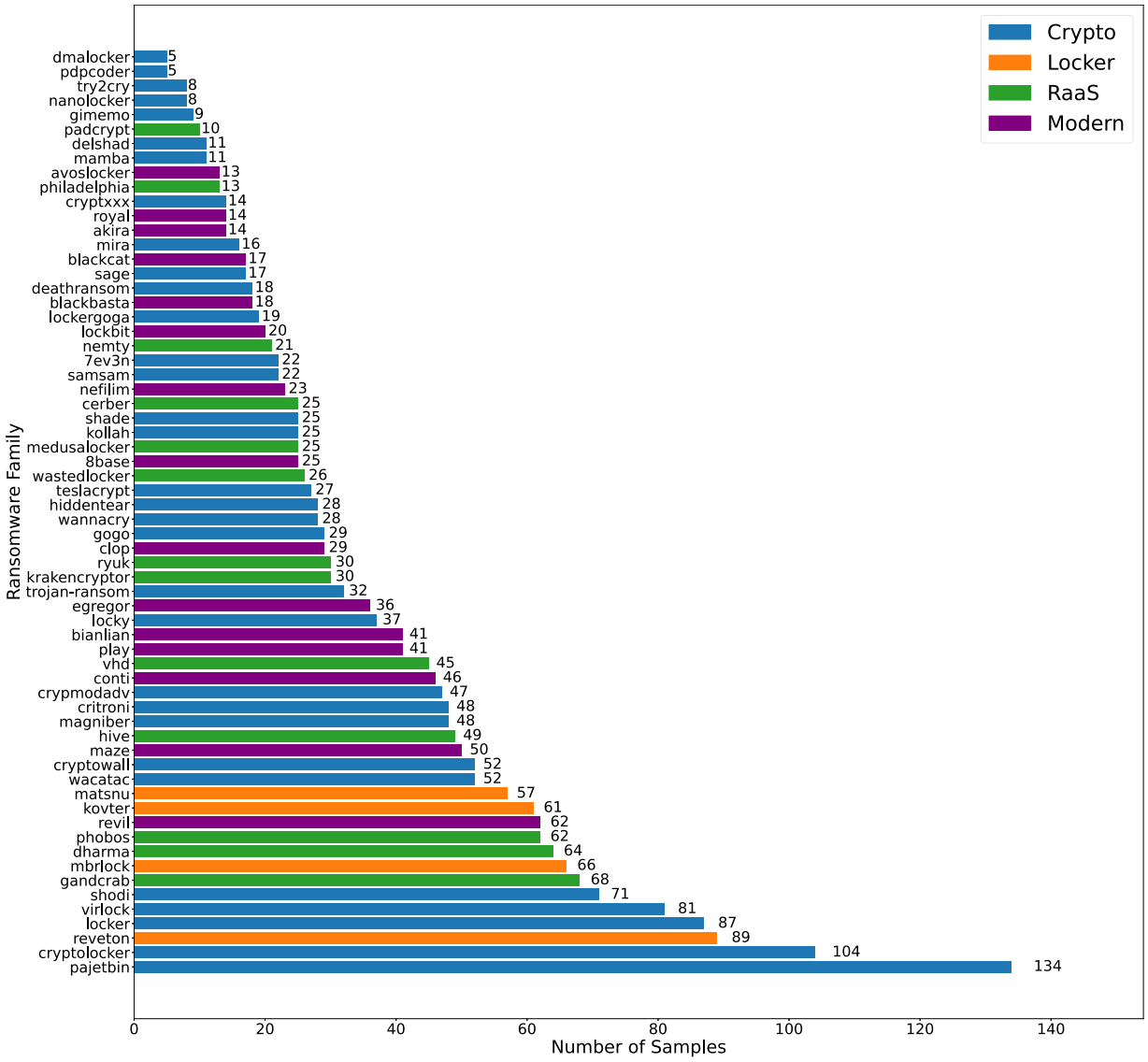


Fig. 3. Distribution of ransomware families, colour-coded by their respective ransomware types. The numbers on the bars represent the number of samples from each ransomware family found in the MLRan Dataset. The dataset includes a total of 64 ransomware families, classified into four categories, as shown in the legend and colour-coded in the bars: 32 families belong to the Crypto type, 15 are Modern, 13 are RaaS, and 4 are Locker.

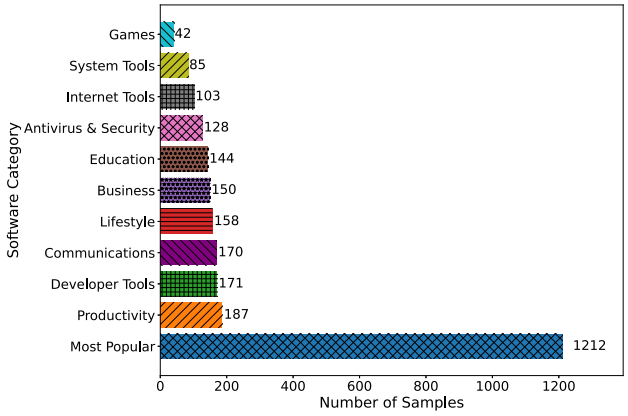


Fig. 4. Distribution of goodware sample categories in the MLRan Dataset. The goodware samples contain 11 categories, with the Most Popular category having the highest sample count of 1212 samples, representing 47.53% of the total. The Productivity category follows with 187 samples, accounting for 7.33%. The exact number of samples for each category is displayed on the corresponding bars.

However, from 2012 onwards, a steady increase in ransomware samples was observed, whereas the number of goodware samples remained relatively stable. Notably, a significant surge in ransomware samples occurred in 2020, with an even sharper increase in 2021. This spike is likely attributed to the escalation of cybercrime during the COVID-19 pandemic, which saw a rise in ransomware attacks targeting organisations and individuals worldwide (Baig et al., 2023; Minnaar and Herbig, 2021). The timestamps for the samples were determined by their first submission to VirusTotal, providing a more reliable temporal reference. This method is more robust than relying on creation dates, which malware authors frequently manipulate to evade detection (Jiang et al., 2024).

Furthermore, additional contextual and metadata information satisfying criterion (C12), including sample hashes, descriptions, and other relevant attributes, is available in the corresponding CSV files within our repository.[20]

[20] MLRan Metadata: https://github.com/faithfulco/mlran/tree/main/2_collected_samples_metadata.

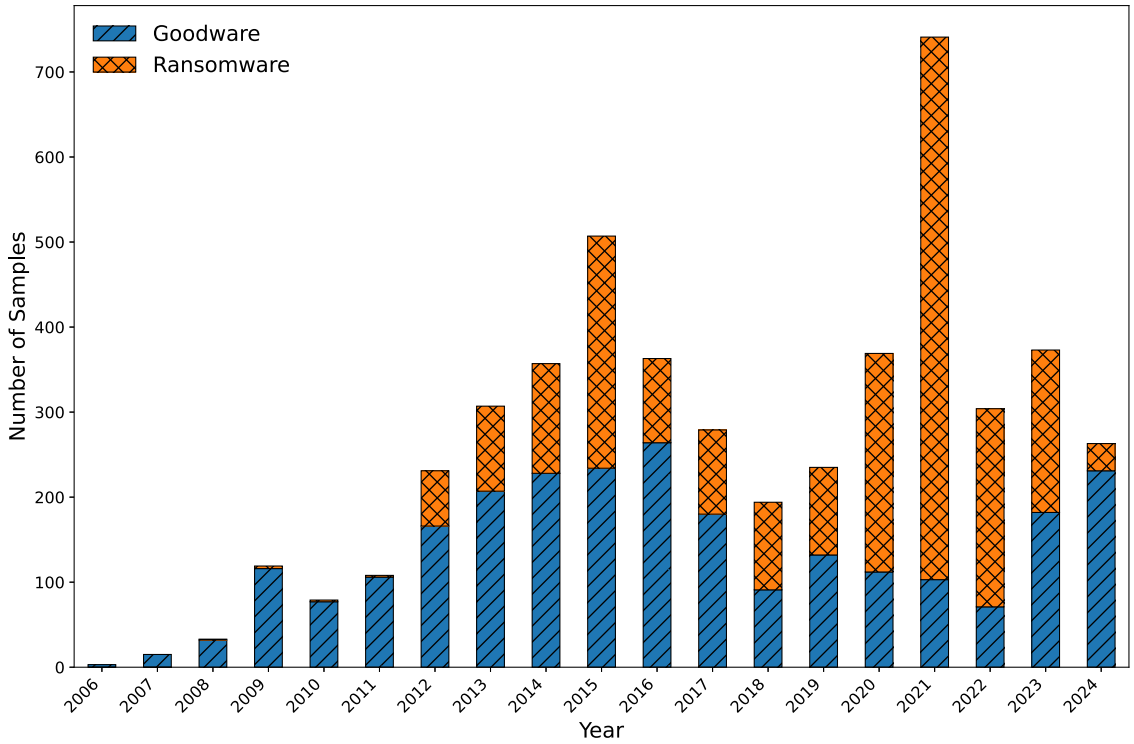


Fig. 5. The stacked bar chart illustrates the distribution of samples in the MLRan dataset across different years, based on their first submission to VirusTotal, segmented by sample type (Goodware and Ransomware). The first submission timestamp from VirusTotal was used as it provides more reliable temporal information compared to creation dates, which are often manipulated by malware authors.[21] Between 2006 and 2011, the dataset predominantly consisted of Goodware samples. Starting in 2012, a steady increase in Ransomware samples was observed, while Goodware samples remained relatively stable, except for a notable spike in Ransomware during 2020 and particularly in 2021, likely driven by the surge in cybercrime during the COVID-19 pandemic.

4.2. Sample collection

Fig. 6 illustrates the methodology used for ransomware sample collection, outlining the steps taken to ensure the ransomware samples' diversity, representativeness, uniqueness, and authenticity. The dataset comprises samples from four primary sources, each subjected to rigorous validation procedures. We developed custom scripts to automate the downloading of both ransomware and goodware samples. These scripts are publicly available and properly documented in our repository.[22]

The first source is Elderan (Sgandurra et al., 2016), a dataset published in 2016 containing 582 ransomware samples with their respective names obtained by manually clustering each ransomware into a well-established family name. We downloaded these samples from VirusShare using a custom-designed script. We verified their hashes against VirusTotal to confirm their authenticity, ensuring each sample had a non-zero detection score. Additionally, duplicate samples were identified and removed to maintain dataset integrity.

The second source is MOTIF (Joyce et al., 2023), a malware reference dataset with ground truth family labels. MOTIF addresses the issue of noisy labels in malware analysis by curating samples based on hundreds of open-source threat intelligence reports published by reputable cybersecurity organisations. From the 3095 samples spanning 502 malware families, we filtered those classified as ransomware, yielding 549 samples across 103 families. Using their hashes, we attempted to retrieve the corresponding binaries from VirusTotal, successfully obtaining 443 samples across 34 families. After duplicate removal and ensuring that VirusTotal detections remained non-zero, the sample count remained unchanged. However, dynamic analysis conducted using the Cuckoo sandbox successfully processed 426 ransomware samples spanning 34 distinct families.

The third source is MarauderMap (Hou et al., 2024), which contains 7796 malicious samples but lacks family labels. To categorise these samples, we applied AVClass (Sebastián et al., 2016) for automatic family classification. We then conducted a manual verification process to ensure that only ransomware samples were retained, resulting in 627 samples across 25 ransomware families. We downloaded these samples from GitHub, and after removing duplicates and ensuring a non-zero VirusTotal detection score, we obtained 621 unique samples. Following analysis in the Cuckoo sandbox, 575 ransomware samples were successfully processed without errors, representing 25 distinct families.

The fourth source consists of ransomware samples curated by the authors, following a methodology similar to that of Joyce et al. (2023). We collected ransomware sample hashes from open-source threat intelligence reports containing detailed ransomware analyses published by reputable cybersecurity organisations. Additionally, we sourced samples from various malware repositories, including MalwareBazaar, VirusShare, VirusTotal, and Hybrid-Analysis. The sample size remained unchanged after removing duplicates and verifying non-zero VirusTotal detections. However, dynamic analysis using the Cuckoo sandbox yielded successful results for only 777 ransomware samples, covering 37 unique families.

Finally, we standardised family names to ensure consistency across sources, accounting for aliasing and duplicate families that appeared in multiple sources. This alignment process improved dataset integrity and ensured uniform classification of ransomware families.

Fig. 7 illustrates the methodology employed for goodware sample collection, detailing the steps taken to ensure diversity, representativeness, uniqueness, and authenticity. We retrieved all goodware samples

[21] VirusTotal Files Info.: https://docs.virustotal.com/reference/files.

[22] MLRan Sample Collection Scripts: https://github.com/faithfulco/mlran/tree/main/1_sample_collection_scripts.

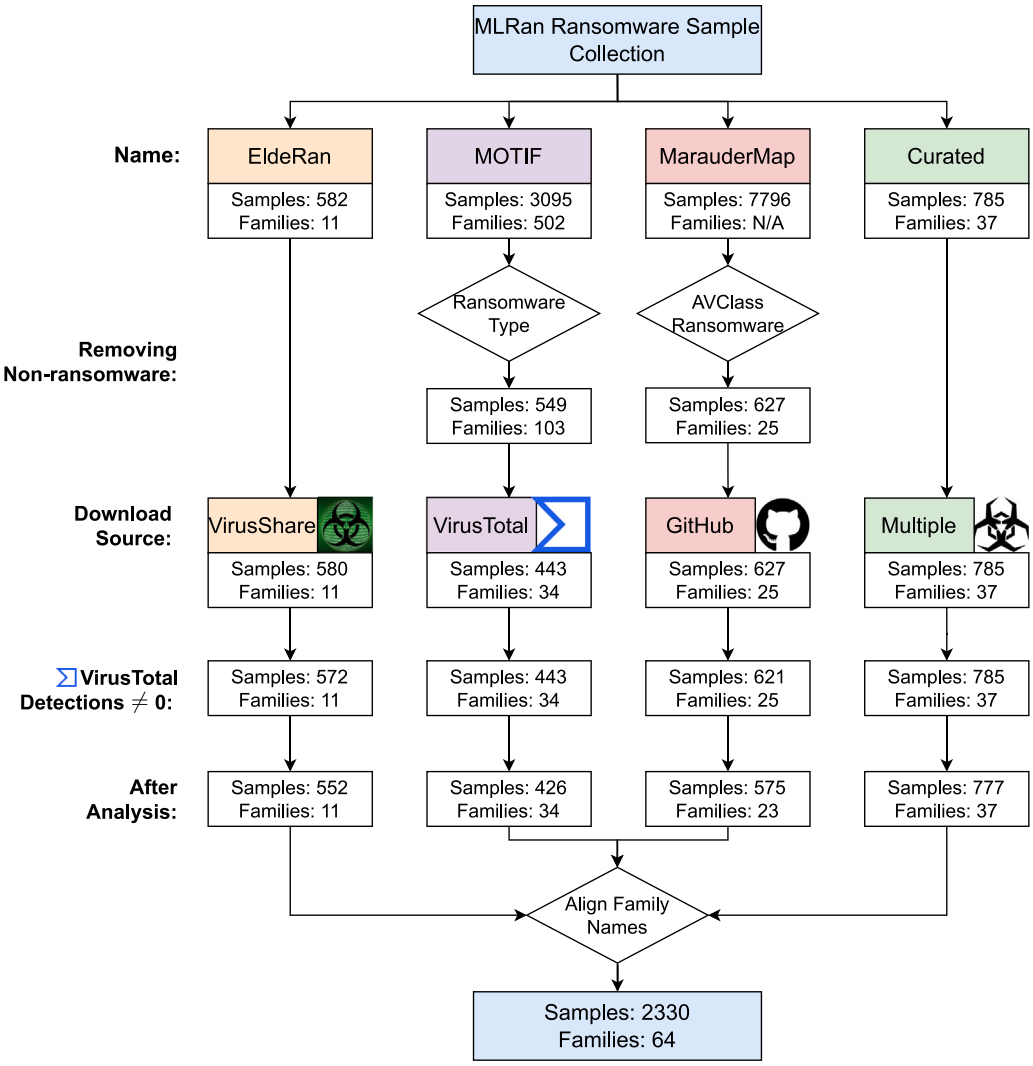


Fig. 6. MLRan ransomware sample collection methodology. The diagram illustrates the four primary sources from which ransomware samples were obtained for the MLRan dataset: EldeRan (552 samples), MOTIF (426 samples), MarauderMap (575 samples), and a curated collection (777 samples).

from Software Informer using a custom-designed script we developed, resulting in an initial dataset of 5010 samples spanning 11 categories. Following the removal of duplicate entries, the dataset included 3837 unique samples.

As highlighted by Botacin et al. (2021), a common pitfall in malware research is the assumption that crawled applications are inherently benign without proper validation. To mitigate this issue, we verified the legitimacy of the goodware samples by submitting their hashes to VirusTotal for analysis. Only those samples that received zero detections across all antivirus engines (about 45–80 engines) were retained, yielding a refined dataset of 2696 goodware samples. Finally, dynamic analysis using the Cuckoo sandbox successfully processed 2550 samples, ensuring coverage across all 11 categories.

4.3. Sample analysis

To ensure accurate and reliable behavioural analysis of ransomware and benign software, we employed **Cuckoo Sandbox**, an open-source automated malware analysis system. Cuckoo allows executing suspicious files in an isolated environment, monitoring their behaviour and generating detailed reports (Liu et al., 2014). This capability is essential for ransomware analysis, as it enables the observation of encryption activities, network communications, and persistence mechanisms in a

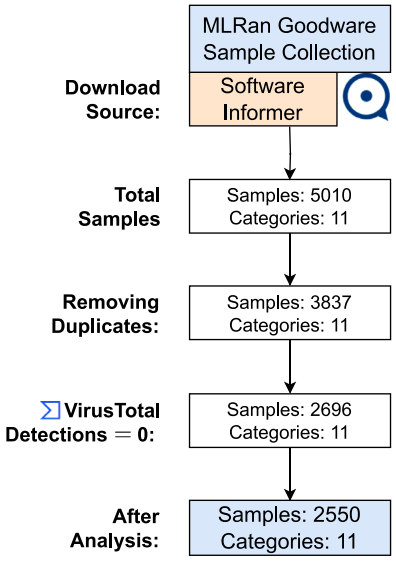


Fig. 7. MLRan goodware sample collection methodology. All the goodware samples were downloaded from the Software Informer website and they cut across 11 different categories.

controlled setting. Given its adaptability, effectiveness and popularity, Cuckoo was selected as the primary analysis tool.

We configured the sandbox environment to mimic a real-world system closely. We installed VMware Workstation on a Windows host machine and deployed Ubuntu 18.04 as the base operating system. We installed Cuckoo Sandbox v2.0.7 alongside a Windows 7 virtual machine, which served as the execution environment for ransomware and goodware samples. This configuration balanced compatibility with modern malware while ensuring a stable, reproducible analysis environment.

Since sandbox-aware malware can detect and evade execution within virtualised environments (Liu et al., 2014), we applied several countermeasures. To enhance realism, we allocated four CPUs, 4 GB of RAM, and sufficient disk space to the virtual machine. Additionally, we installed commonly used software, including Adobe PDF Reader, .NET Framework, Java, Flash, Visual C++ Redistributable (vcredist 2015u3), and Internet Explorer 11, using VMCloak to automate installation. This ensured the virtual machine resembled a genuine user system.

To further prevent detection, we integrated Cuckoo's *Human Auxiliary Module*,[23] which simulates user interactions such as keyboard input and mouse movements. The module randomises cursor movements and speeds to mimic human behaviour, reducing the likelihood of detection by sandbox-aware malware.

Another auxiliary module we integrated into the analysis environment is *Disguise Auxiliary Module*,[24] which enhances realism by modifying system attributes to evade malware detection. Specifically, the module randomises the Windows Product ID, modifies SCSI identifiers to replace virtual machine-related strings with realistic hardware names, and alters BIOS information, including system and video BIOS dates and versions. Additionally, it patches ACPI tables to replace VirtualBox-related identifiers with manufacturer names, modifies processor details to appear as physical hardware, and changes manufacturer and product names in system information. It also updates hard drive paths to resemble real hardware configurations. Additionally, to counteract time-based evasion techniques, where ransomware delays execution to avoid detection, we set an analysis timeout of 120 s to capture delayed malicious behaviour. These modifications prevent ransomware from identifying the environment as a sandbox, further improving the reliability of behavioural analysis.

4.4. Feature extraction

As summarised in Table 3, the feature extraction methodology in this study focuses on nine categories derived from Cuckoo Sandbox reports, i.e. API calls, registry key operations, file operations, directory operations, extracted strings, network activity, system resource usage, dropped files extensions and types, and behavioural signatures. These features capture key behavioural aspects of software and are represented using a **binary presence approach**, where each feature is assigned a value of 1 if present or 0 if absent. This method aligns with prior research (Sgandurra et al., 2016), ensuring consistency in malware analysis.

We followed the approach in Algorithm 1 to extract API calls from Cuckoo Sandbox JSON reports. The algorithm loads each report (Lines 4–5), extracts the apistats section (Line 6), and creates a dictionary where each observed API call is recorded with a binary indicator prefixed by API: (Lines 7–10). Sample IDs are extracted from filenames and used to index the data (Line 11). All records are stored and converted into a DataFrame (Line 13), with missing values filled as zeros (Line 14) and sorted by sample ID (Line 15). The final DataFrame represents each sample as a vector of API call presences (Line 16).

[23] Cuckoo Human Module: https://github.com/cuckoosandbox/cuckoo/blob/master/cuckoo/data/analyzer/windows/modules/auxiliary/human.py.

[24] Cuckoo Disguise Module: https://github.com/cuckoosandbox/cuckoo/blob/master/cuckoo/data/analyzer/windows/modules/auxiliary/disguise.py.

Algorithm 1: Processing Cuckoo Sandbox JSON reports for API calls

```
Input  : Folder reports_folder containing Cuckoo JSON reports
Output: DataFrame df summarizing API calls per sample
1  Initialize empty list data to store API call summaries;
2  Initialize empty list file_ids to store report IDs;
3  Retrieve all filenames in reports_folder matching digits or ending with .json;
4  foreach filename in all_files do
5      Load JSON file filename into report;
6      Extract apistats section from report.behavior.apistats;
7      Initialize empty dictionary summary for API calls;
8      foreach api_dict in apistats.values() do
9          foreach api in api_dict.keys() do
10             summary["API:api"] ← 1;
11     Extract file_id from filename (removing extension if necessary);
12     Append summary to data and file_id to file_ids;
13 Create DataFrame df from data, indexed by file_ids;
14 Fill missing values in df with 0 and convert to integers;
15 Sort df by sample_id in ascending order;
16 return df
```

5. Experimental design

This section presents our approach to conducting the experiments demonstrating the effectiveness of the MLRan dataset. It illustrates the research questions and the process adopted to answer them, the selected feature selection techniques and ML models, and the evaluation metrics we used to evaluate ransomware detection performance.

5.1. Research questions

In this study, we aim to answer the following research questions:

- RQ1: How effectively do traditional ML classifiers distinguish ransomware from goodware using the MLRan dataset?
- RQ2: How do feature selection techniques impact the performance of ransomware detection models on the MLRan dataset?
- RQ3: Which features in the MLRan dataset are most predictive for ransomware detection?
- RQ4: How can analysing misclassified decisions reveal model weaknesses, enhance interpretability, and improve the robustness of ransomware detection models?

5.2. Feature selection techniques

We employed feature selection (FS) techniques to ensure unique feature names, eliminate duplicate entries, and remove constant features lacking variance. Feature selection identifies and selects the most relevant features from a dataset, reducing dimensionality while preserving the model's decision-making quality (Zebari et al., 2020). Eliminating redundant or irrelevant variables simplifies models, improves computational efficiency, enhances model interpretability, reduces overfitting, and ultimately increases the accuracy and generalisability of ML algorithms (Theng and Bhoyar, 2024).

Feature selection methods fall into four categories: filter, wrapper, embedded methods, and genetic algorithms (Kamolov, 2021). Filter methods using measures like information gain or χ^2-statistics to evaluate features independently are the most straightforward and efficient. Wrapper methods evaluate feature subsets by training a model, often

Table 3
Summary of extracted feature categories from Cuckoo Sandbox reports. Each category represents a key behavioural aspect of the analysed software. The table details the feature descriptions, their extraction sources within the Cuckoo report, and example feature names.

Feature category	Description	Cuckoo report section	Example features names
API Calls (API)	Captures system interactions related to file manipulation, memory access, network activity, and process execution. These calls indicate how a program interacts with the OS.	`behaviour → apistats`	`API:GetAdaptersInfo`, `API:CreateProcessInternalW`, `API:CryptUnprotectData`, `API:DnsQuery_UTF8`
Registry Keys (REG)	Tracks registry modifications, including keys opened, deleted, read, or written. These modifications are often used for persistence and disabling security features.	`behaviour → summary → regkey_opened, regkey_deleted, regkey_read, regkey_written`	`REG:DELETED:HKEY_CLASSES_ROOT\*\shell \Secure Eraser`, `REG:WRITTEN:\REGISTRY \USER \.DEFAULT\SOFTWARE \Piriform \Recuva \Language`
File Operations (FILE)	Monitors file access patterns, including creation, deletion, opening, and writing, which can be indicative of ransomware encryption attempts.	`behaviour → summary → file_created, file_deleted, file_written, file_opened`	`FILE:CREATED:c:\!!!how_to_decrypt!!! .txt`, `FILE:WRITTEN:z:\boot \recovery _instructions.html`
Directory Operations (DIR)	Identifies directory creation and enumeration, which ransomware often uses to access and encrypt multiple files.	`behaviour → summary → directory_created, directory_enumerated`	`DIRECTORY:CREATED:\.\c :\programdata`, `DIRECTORY:ENUMERATED:z:\boot \sv -se\*`
Strings (STR)	Extracts human-readable text from binaries, including error messages, function names, registry paths, and command-line arguments, providing contextual clues to software intent.	`strings`	`STRING:snd_clipcopy`, `STRING:m\device \harddiskvolume 2\program files\mcafee \engine \avvclean .datp`
Network Activity (NET)	Tracks network communication, such as IP connections, domain resolutions, and host interactions, which are crucial for identifying C2 communications.	`network → connects_ip, connects_host, resolves_host`	`NETWORK:CONNECTS_HOST:104.131.182.103`, `NETWORK:RESOLVES_HOST:yahoo.com`
System Resources (SYS)	Examines how software interacts with system resources, such as DLLs, command-line execution, mutex creation, and GUID usage.	`behaviour → summary → dll_loaded, command_line, mutex, guid`	`SYSTEM:DLL_LOADED:user32.dll`, `SYSTEM:MUTEX: $Mutex_XYZ$`
Dropped Files Extensions and Types (DROP)	Captures files created or modified by the sample, including file types and extensions, to detect ransomware payloads or encrypted file formats.	`dropped`	`DROP:EXTENSION:.exe`, `DROP:TYPE:zip_archive_data`
Signatures (SIG)	Detects predefined behavioural patterns associated with malware, including anti-VM detection and ransomware indicators.	`signatures`	`SIGNATURE:allocates_execute_remote_ process`, `SIGNATURE:antiemu_wine`

giving better results but requiring more computation, such as Recursive Feature Elimination (RFE). Embedded methods combine feature selection with the learning process, usually through regularisation. Genetic algorithms select features by evolving subsets based on their performance, offering a flexible, optimisation-based approach. While wrapper and embedded methods may work better with specific models, filter methods are more consistent across different ones.

This study employs three widely used techniques: Mutual Information (MI), Chi-Square, and RFE. The study develops a two-stage feature selection strategy. The initial features comprise 6.4M sparse binary features obtained by presence–absence vectorisation of dynamic analysis JSON reports, encoding API calls, file and registry operations, strings, mutexes, and network artefacts (Section 4.4). In Stage 1, MI is applied as a model-agnostic statistical filter, with Chi-Square used as a

Table 4
Summary of GUIDE-MLRan and MLRan Dataset Compliance with the Guidelines. It outlines the fifteen criteria of the GUIDE-MLRan framework for constructing high-quality behavioural ransomware datasets. Columns indicate the guideline category, specific criterion (C1–C15), a brief description of the requirement, and a summary of how the MLRan dataset fulfils each criterion.

Category	Criteria (C)	Description	MLRan Compliance Summary
Sample Diversity and Representa-tiveness	C1. Diverse Ransomware Samples	Include varied ransomware types and families (e.g., Locker, Crypto, RaaS, Modern).	Includes 2330 samples across 64 families, covering all major ransomware types (Crypto, RaaS, Locker, Modern), reflecting both legacy and emerging variants.
	C2. Diverse Goodware Samples	Include diverse benign software (e.g., common applications, browsers, word processors, utilities).	Curated 2550 goodware samples across 11 real-world categories, including productivity, developer tools, internet utilities, games, and security software.
	C3. Balanced Class Distribution	Maintain a balanced ratio of ransomware and goodware samples.	Achieves inter-class balance (52.25% goodware, 47.75% ransomware); intra-class diversity captured via broad ransomware family representation.
Sample Quality and Accuracy	C4. Accurate Labelling	Ensure accurate annotation of ransomware and goodware with detailed metadata.	Ground truth family labels and naming using AVClass.
	C5. Sample size	Collect enough samples to capture common ransomware behavioural patterns.	4880 samples sourced from four high-quality repositories (ElderRan, MOTIF, MarauderMap, and curated threat intel); ensures comprehensive, high-fidelity behavioural coverage.
	C6. Covered Time Period	Include both recent and outdated ransomware for real-world applicability.	Samples span from 2006–2024 using VirusTotal first submission timestamps.
Sandbox and Testbed Requirements	C7. Sandbox Hardening	Simulate real-world user behaviour to bypass anti-sandbox techniques.	Cuckoo sandbox hardened using disguise and human activity simulation modules, extended timeouts, and realistic system configurations for evasion countermeasures.
	C8. Comprehensive Behavioural Analysis	Capture dynamic and static features for a comprehensive behavioural profile.	Extracts nine dynamic behavioural feature groups (API, REG, FILE, DIR, STR, NET, SYS, DROP, SIG) from Cuckoo reports for robust malware profiling.
Representative Feature Extraction and Modelling	C9. Relevant Feature Extraction	Extract features such as API calls, file operations and network behaviours relevant to detection.	Publicly available scripts extract critical features from structured JSON reports; features are relevant to file, system, and network activity.
	C10. Data Preprocessing	Clean, normalise, and standardise data to ensure quality and consistency.	Preprocessing includes duplicate removal, standardisation, binary encoding, and metadata alignment for consistent ML input.
	C11. Model Training and Evaluation	Ensure robust model training and evaluation.	Models trained using rigorous protocol to avoid test/temporal/ selective snooping bias; evaluation incorporates several performance evaluation metrics.
Documentation, Reproducibility, and Data Extension	C12. Availability of Contextual and Metadata Information	Provide detailed metadata about samples, datasets, sandboxes and methodologies.	Comprehensive metadata provided (hashes, types, timestamps, labels, source); sandbox configuration and feature definitions clearly documented.
	C13. Public Availability of Data and Code	Share datasets and codes with documentation to encourage reproducibility and benchmarking.	All Cuckoo parsers, automation scripts, and dataset metadata made available on GitHub with documentation and version control.
	C14. Ethical and Legal Considerations	Ensure anonymisation, privacy, and compliance with regulations.	Anonymisation ensured; only metadata and derived features shared; binaries handled securely and access-controlled to prevent misuse.
	C15. Continuous Updates	Regularly update datasets to include emerging ransomware threats.	Dataset incorporates ransomware samples up to 2024; framework enables longitudinal extension and collaborative community curation.

Table 5
Performance comparison of machine learning models for ransomware and goodware detection using three feature selection (FS) techniques: Mutual Information (MI) with thresholds of 0.01 and 0.001, and Chi-Square. The table presents accuracy (Acc), balanced accuracy (Bal. Acc), precision (Pre), recall (Re), F1 score (F1), and computation Time (sec) for each model: XGBoost, Decision Trees, Logistic Regression, and Random Forest. Results show that logistic regression with MI (0.01) outperforms other FS methods in terms of performance and computational efficiency.

FS	Model	Acc	Bal. Acc	Pre	Re	F1	Time (s)
MI (0.01)	LightGBM	94.05	94.07	94.06	94.05	94.05	7876.11
	Decision Trees	97.54	97.59	97.57	97.54	97.54	10.76
	Logistic Regression	97.54	97.59	97.57	97.54	97.54	4.75
	Random Forest	97.03	97.10	97.09	97.03	97.03	13.41
	Extra Trees	97.33	97.38	97.37	97.33	97.33	4.98
MI (0.001)	LightGBM	94.56	94.56	94.56	94.56	94.56	24797.11
	Decision Trees	96.62	96.67	96.65	96.62	96.62	50.89
	Logistic Regression	98.05	98.11	98.05	98.05	98.05	19.97
	Random Forest	96.92	97.00	96.99	96.92	96.92	37.40
	Extra Trees	96.92	96.99	96.98	96.92	96.92	20.68
Chi-Square	LightGBM	93.84	93.85	93.85	93.85	98.41	23017.73
	Decision Trees	96.82	96.86	96.85	96.82	96.82	115.36
	Logistic Regression	98.05	98.11	98.09	98.05	98.05	120.10
	Random Forest	93.54	93.79	94.13	93.54	93.53	968.32
	Extra Trees	93.54	93.80	94.17	93.54	93.53	833.34

baseline for comparison within each feature group. The MI threshold of 0.01 is selected empirically based on comparisons with MI(0.001) and Chi-Square (Table 5), where lower thresholds retain substantially more features without consistent gains in balanced accuracy and incur higher computational cost. MI(0.01) reduces the feature space by over 99.6%, from 6.4M to 24,162 features, while preserving comparable performance. In Stage 2, RFE is applied to the MI-filtered feature set, with balanced accuracy peaking when retaining 2% of the Stage 1 features, corresponding to 483 features (Fig. 9), demonstrating that the final dimensionality results from performance-guided optimisation rather than ad hoc thresholding.

5.2.1. Mutual Information (MI)

Mutual Information quantifies the dependency between a feature X and the target variable Y, measuring the reduction in uncertainty about Y given X:

$$I(X;Y) = \sum_{x\in X}\sum_{y\in Y} P(x,y)\log\frac{P(x,y)}{P(x)P(y)} \tag{1}$$

A higher $I(X;Y)$ value indicates stronger feature relevance. MI is non-parametric and captures both linear and nonlinear relationships.

5.2.2. Chi-Square (χ^2) test

The χ^2 test evaluates the statistical dependence between a categorical feature and the target variable, computed as:

$$\chi^2 = \sum_{i=1}^{n}\frac{(O_i - E_i)^2}{E_i} \tag{2}$$

where O_i and E_i denote the observed and expected frequencies, respectively. A high χ^2 score suggests strong dependence, making the feature valuable for classification.

5.2.3. Recursive Feature Elimination (RFE)

RFE iteratively removes the least important features based on model coefficients (e.g., β in linear models) or feature importance scores from tree-based models. The process refines feature selection by retaining those with the highest predictive contribution.

5.3. Machine learning models

Machine learning models vary in complexity and interpretability, offering different performance and computational efficiency trade-offs. This study employs Light Gradient Boosting Machine (LightGBM), Decision Trees, Logistic Regression, and Random Forest, each with distinct advantages in ransomware detection.

5.3.1. Light gradient boosting machine (LightGBM)

LightGBM is a gradient boosting framework that constructs decision trees using a leaf-wise growth strategy, which selects the leaf with the maximum loss reduction to split. This approach often leads to faster convergence and better accuracy than level-wise methods. LightGBM is optimised for high performance through histogram-based binning, gradient-based one-side sampling (GOSS), and exclusive feature bundling (EFB). The *objective function* follows the regularised gradient boosting formulation:

$$\mathcal{L}(\Theta) = \sum_{i=1}^{n} l(y_i,\hat{y}_i) + \lambda\sum_{k=1}^{K}\|f_k\|^2 \tag{3}$$

where $l(y_i,\hat{y}_i)$ is the differentiable loss function, and the regularisation term $\|f_k\|^2$ controls tree complexity. LightGBM is particularly effective for large-scale and high-dimensional learning tasks such as ransomware classification.

5.3.2. Decision trees

A Decision Tree splits data using conditions at nodes, forming a tree structure for decision-making. It uses measures like *Gini impurity*:

$$G = 1 - \sum_{i=1}^{C} p_i^2 \tag{4}$$

where p_i is the probability of a class. Decision Trees are interpretable but prone to overfitting.

5.3.3. Logistic regression

Logistic Regression models the probability of a binary outcome using the *sigmoid function*:

$$P(Y=1|X) = \frac{1}{1+e^{-(\beta_0+\sum_{i=1}^{n}\beta_i x_i)}} \tag{5}$$

where β_0 is the intercept and β_i are feature coefficients. It is computationally efficient but limited to linearly separable data.

5.3.4. Random forest

Random Forest is a bagging ensemble of Decision Trees, reducing overfitting by averaging multiple tree predictions. The final classification is determined via *majority voting*:

$$\hat{y} = \text{mode}(y_1, y_2, \ldots, y_T) \tag{6}$$

where y_T is the prediction from the Tth tree. For regression, predictions are averaged:

$$\hat{y} = \frac{1}{T}\sum_{t=1}^{T} y_t \tag{7}$$

It handles high-dimensional data effectively but is computationally expensive.

5.3.5. Extra trees (Extremely randomised trees)

Extra Trees is an ensemble method that builds multiple unpruned decision trees using randomly selected features and split thresholds. Unlike Random Forests, it increases randomness to reduce variance and enhance generalisation. Predictions are aggregated as:

$$\hat{y} = \frac{1}{T}\sum_{t=1}^{T} f_t(x) \tag{8}$$

where $f_t(x)$ is the prediction from the tth tree. Extra Trees is efficient and effective for high-dimensional tasks such as ransomware detection.

5.4. Evaluation metrics

To assess model performance in ransomware detection, we employed the following evaluation metrics:

5.4.1. Accuracy

Accuracy measures the proportion of correctly classified instances:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{9}$$

where TP and TN represent true positives and true negatives, respectively, while FP and FN denote false positives and false negatives. Accuracy is effective for balanced datasets but may be misleading in imbalanced scenarios.

5.4.2. Balanced accuracy

Balanced Accuracy adjusts for class imbalances by averaging the recall of each class:

$$\text{Balanced Accuracy} = \frac{1}{2}\left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP}\right) \tag{10}$$

This metric ensures a fair evaluation when class distributions are skewed.

5.4.3. Precision

Precision quantifies the proportion of correctly identified positive instances out of all predicted positives:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{11}$$

A high precision indicates fewer false positives, which is critical in security applications where misclassifying benign software as ransomware has significant consequences.

5.4.4. Recall

Recall, or sensitivity, measures the model's ability to detect positive instances:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{12}$$

A high recall ensures fewer false negatives, making it essential for accurately identifying ransomware threats.

5.4.5. F1-score

The F1-Score balances precision and recall, providing a single performance metric:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{13}$$

It is particularly useful when false positives and false negatives need equal consideration.

5.5. Ransomware detection pipeline

To answer the research questions, we performed the activities in the ransomware detection pipeline presented in Fig. 8. First, we conducted the dynamic analysis of the collected ransomware and goodware samples using Cuckoo Sandbox, which generates JSON reports capturing runtime behaviour. Then, we used a custom Python-based extractor to parse these reports to generate over 6.4 million binary features, grouped into nine behavioural categories, including API calls, registry activity, file operations, and network behaviour.

A **time-aware 80:20 train–test split** is employed to ensure temporal consistency and realistic evaluation. Samples are grouped by type and chronologically ordered using their first submission date to VirusTotal, with the earliest 80% used for training and the most recent 20% reserved for testing. This design prevents temporal leakage and reflects real-world deployment, where detection models are trained on historical data and applied to future, previously unseen samples. Prior work has shown that random or k-fold cross-validation can yield optimistically biased performance estimates in malware classification due to family overlap and concept drift (Pendlebury et al., 2019; Allix et al., 2015). Enforcing temporal separation is therefore essential for assessing generalisation under evolving ransomware behaviours.

Then, we performed feature selection in two stages. First, we conducted group-wise mutual information filtering (threshold = 0.01), which reduced the feature space from 6.4 million to 24,162. Then we performed RFE, which selects the most informative 483 features. We then trained a classification model on these features and evaluated it on the test set.

To avoid temporal leakage, hyperparameter tuning is performed exclusively on the training set (the earliest 80% of samples). Specifically, for each model, we use `GridSearchCV` on `X_train, y_train` with 3-fold cross-validation and select hyperparameters using ROC–AUC for binary classification and weighted F1 for multiclass classification. The temporally held-out test set (the most recent 20% of samples) is not used during tuning or model selection and is accessed only for final performance reporting. The model hyperparameters and justifications for each algorithm used in the study, including their value ranges and rationales based on empirical or theoretical considerations, are provided in Table 9.

To enhance model transparency, we applied SHAP and LIME to explain individual predictions and highlight influential features, providing interpretable insights into the behavioural traits that distinguish ransomware from goodware.

6. Results

This section presents the results we obtained to answer the research questions: performance of traditional ML classifiers (baselines) (RQ1), the impact of feature selection on the performance of ransomware detection models (RQ2), feature importance (RQ3), analysis of model decisions, and explanations of misclassified instances at both global and local levels (RQ4).

6.1. Baseline performance (RQ1)

We compared the performance of the baseline ML models for distinguishing ransomware from goodware using three feature selection techniques.Table 5 presents our comparative analysis. We consider three FS methods as part of stage 1 (MI with thresholds of 0.01 and 0.001, and Chi-Square) highlighting trade-offs between model performance and computational cost. While logistic regression achieves marginally higher accuracy and F1 score with MI at a 0.001 threshold, MI at 0.01 is selected as the preferred method due to its ability to retain significantly fewer features while delivering comparable performance (97.54% accuracy and F1 score). Notably, it also achieves the lowest computation time (4.75 s), making it the most efficient choice. This

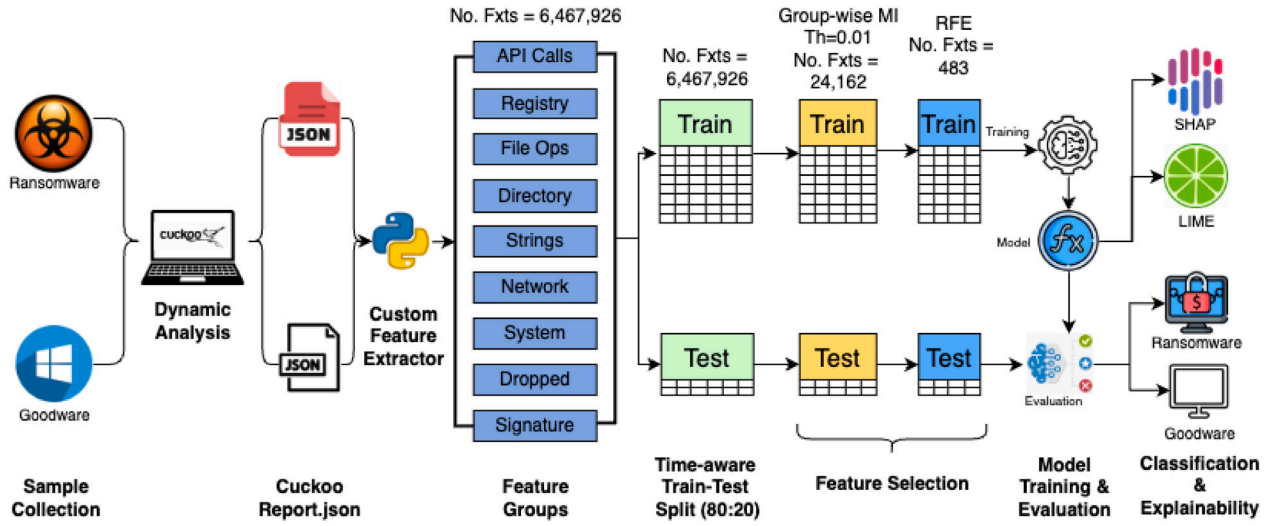


Fig. 8. Ransomware detection pipeline showing dynamic analysis, custom feature extraction, multi-stage feature selection, model training, and explainability using SHAP and LIME.

threshold reduces the feature space from 6,467,926 to 24,162, enhancing model interpretability and reducing complexity without compromising predictive power. Overall, MI-based feature selection consistently outperforms Chi-Square, and logistic regression emerges as the most effective and computationally efficient model. In contrast, LightGBM performs the least favourably across all FS methods, particularly with MI (0.001), where it exhibits both lower accuracy and significantly higher computation time, indicating poor scalability in this setting. The fact that Logistic Regression outperforms LightGBM may be attributed to the nature of the feature representation rather than a limitation of ensemble methods. After feature selection, the remaining behavioural features may favour regularised linear models that are simpler compared to LightGBM, which favours more non-linear and complex relationships.

Result 1: Baseline performance

Logistic regression with MI (0.01) offers the best trade-off between performance, efficiency, and feature reduction, while LightGBM performs worst due to lower accuracy and high computation cost.

6.2. Impact of feature selection (RQ2)

In Stage 1 of our feature selection pipeline, MI (0.01) yielded an initial feature set of 24,162. In Stage 2, this set was reduced to 483 features using RFE as described in Algorithm 2, with logistic regression (the best-performing model from Stage 1) as the base estimator. The algorithm iteratively evaluates performance across varying feature counts: it initialises a logistic regression model (Lines 1–2), computes feature counts from selection percentages (Lines 5–6), applies RFE (Lines 11–14), trains and tests the model (Lines 15–18), and records balanced accuracy (Lines 19–21). Results are stored and returned in a dictionary (Lines 22–25). Fig. 9 shows that the highest balanced accuracy (98.07%) is achieved using just 2% of the features (483), highlighting the effectiveness of the proposed feature selection technique.

Table 6 presents the performance of the models on the selected 483 features across three ransomware detection tasks: binary, multi-class (type), and family classification. Logistic Regression delivers strong, consistent results with high accuracy and minimal computation time, particularly in the binary and family classifications. Extra trees model performs best in ransomware type classification. LightGBM shows lower balanced accuracy on the binary and family classification tasks.

Table 7 shows the number of features selected in each feature group at each stage of the two stage feature selection technique. In the **API** feature group, the proportion of selected features increases significantly from 0.0048% in the original dataset to 0.5670% after MI(0.01) and further to 13.0434% after RFE, highlighting the increasing importance of API features. Similarly, for **REG** (Registry Keys), the proportion

Algorithm 2: Perform Recursive Feature Elimination and evaluate logistic regression performance

Input: X_{train}: Training features, X_{test}: Testing features, y_{train}: Training labels, y_{test}: Testing labels
Output: A dictionary mapping percentages to balanced accuracy scores

1 **Step 1: Initialise logistic regression model**
2 Set $model \leftarrow$ LogisticRegression(max_iter=10000,
3 solver='saga', random_state=42)
4 **Step 2: Define feature selection percentages**
5 Set $percentages \leftarrow [1, 2, 3, 4, 5, 10, 20, 50, 70, 90]$
6 Set $feature_counts \leftarrow [\lfloor (p/100) \cdot |X_{\text{train}}| \rfloor$ for $p \in percentages]$
7 **Step 3: Initialise balanced accuracies dictionary**
8 Set $balanced_accuracies \leftarrow \{\}$
9 **Step 4: Recursive Feature Elimination (RFE)**
10 **foreach** $num_features \in feature_counts$ **do**
11 **Perform RFE for** $num_features$ **features:**
12 Set $rfe \leftarrow$ RFE(estimator=model,
13 n_features_to_select=num_features, step=0.1)
14 Fit RFE: $rfe.fit(X_{\text{train}}, y_{\text{train}})$
15 **Transform test set:**
16 Set $X_{\text{test_transformed}} \leftarrow rfe.transform(X_{\text{test}})$
17 **Fit logistic regression on selected features:**
18 Fit $model$ using $rfe.transform(X_{\text{train}})$ and y_{train}
19 **Calculate balanced accuracy:**
20 Set $y_{\text{pred}} \leftarrow model.predict(X_{\text{test_transformed}})$
21 Set $balanced_acc \leftarrow$ balanced_accuracy_score$(y_{\text{test}}, y_{\text{pred}})$
22 Add to dictionary:
$balanced_accuracies[num_features] \leftarrow balanced_acc$
23 **end**
24 **Step 5: Return balanced accuracies dictionary**
25 **return** $balanced_accuracies$

Table 6
Performance comparison of machine learning models for ransomware classifications across the three labels using the features from RFE. The table presents accuracy (Acc), balanced accuracy (Bal. Acc), precision (Pre), recall (Re), F1 score (F1), and computation Time (sec) for each model: XGBoost, Decision Trees, Logistic Regression, and Random Forest.

FS	Model	Acc	Bal. Acc	Pre	Re	F1	Time (s)
Binary	LightGBM	94.05	94.02	94.05	94.05	94.05	1206.58
	Decision Trees	93.54	93.43	93.61	93.54	93.53	12.13
	Logistic Regression	98.15	98.18	98.16	98.15	98.15	0.86
	Random Forest	96.62	96.63	96.62	96.62	96.62	2.21
	Extra Trees	97.13	97.18	97.16	97.13	97.13	1.60
Types	LightGBM	81.95	73.43	86.61	82.34	81.47	5447.46
	Decision Trees	83.79	74.43	84.43	83.79	83.78	12.17
	Logistic Regression	86.46	79.86	87.70	86.46	86.53	1.31
	Random Forest	86.67	79.34	87.43	86.67	86.65	2.67
	Extra Trees	87.69	80.71	88.32	87.69	87.77	1.61
Families	LightGBM	74.15	30.42	69.06	74.15	68.07	7132.76
	Decision Trees	75.38	41.66	80.84	75.38	76.72	1.79
	Logistic Regression	81.85	47.07	83.70	81.85	81.35	2.24
	Random Forest	78.77	45.55	80.82	78.77	77.84	2.95
	Extra Trees	81.13	45.23	83.35	81.13	81.31	1.54

Table 7
Distribution of number of features selected across the feature groups in the MLRan dataset: application programming interface (API), registry keys (REG), file operations (FILE), directory operations (DIR), strings (STR), network activities (NET), system operations (SYS), drop operations (DROP), and signatures (SIG). Feature selection is carried out in two stages: Stage 1 applies mutual information (MI) with a 0.01 threshold to the original features, and Stage 2 uses recursive feature elimination (RFE) on the features selected in Stage 1. N represents the number of features belonging to that group and % represents the percentage of each feature group relative to the total features selected by each technique.

Feature groups	Original		After MI(0.01) FS		After RFE FS	
	N	%	N	%	N	%
API	313	0.0048	137	0.5670	63	13.0434
REG	525 505	8.1248	5369	22.2208	90	18.6335
FILE	2 078 287	32.1322	14 044	58.1243	16	3.3126
DIR	158 123	2.4447	820	3.3938	10	2.0704
STR	3 632 119	56.1559	3347	13.8523	223	46.1697
NET	4814	0.0744	3	0.0124	0	0.0000
SYS	16 911	0.2615	321	1.3285	38	7.8674
DROP	51 651	0.7986	75	0.3104	23	4.7619
SIG	203	0.0031	46	0.1904	20	4.1407
Total	6 467 926		24 162		483	

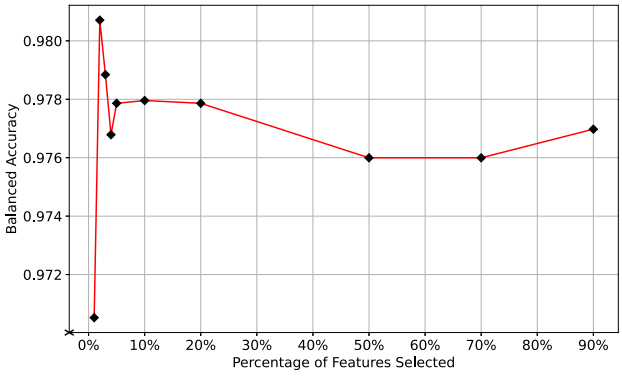


Fig. 9. This plot shows the balanced accuracy of a logistic regression model as a function of the percentage of features selected using Recursive Feature Elimination. The initial dataset contains 24,162 features, and RFE is used to select subsets of features corresponding to different percentages iteratively. The first 5 points on the x-axis are 1%, 2%, 3%, 4%, and 5%. The highest balanced accuracy (98.07%) is achieved when only 2% of the features (483 features) are selected, demonstrating that careful feature selection can improve model performance while reducing computational complexity.

rises from 8.1248% to 22.2208% after MI(0.01), then drops slightly to 18.6335% after RFE, reflecting the selection of more relevant features during the MI(0.01) stage, followed by refinement with RFE.

In the **FILE** feature group, MI(0.01) increases the selected proportion from 32.1322% to 58.1243%, but RFE significantly reduces it to 3.3126%, indicating that RFE retained only the most relevant file-related features. The **DIR** (Directory Operations) group shows a modest increase from 2.4447% to 3.3938% after MI(0.01), followed by a small reduction to 2.0704% after RFE, suggesting less importance placed on directory operations.

The **STR** (Strings) feature group shows a dramatic shift, with the percentage dropping from 56.1559% to 13.8523% after MI(0.01) and then increasing to 46.1697% after RFE, indicating that RFE preserved a significant number of string-based features. For **SYS** (System Operations), the proportion increases from 0.2615% to 1.3285% after MI(0.01) and increases to 7.8674% after RFE. The **DROP** (Drop Operations) group shows a decrease from 0.7986% to 0.3104% after MI(0.01) and a slight increase to 4.7619% after RFE, highlighting RFE's role in retaining important drop operations. **SIG** (Signatures) features increase from 0.0031% to 0.1904% after MI(0.01) and then to 4.1407% after RFE, indicating that only the most critical signature features were retained.

In contrast, **NET** (Network Activities) features were reduced from 0.0744% to 0.0124% after MI(0.01) and completely eliminated (0%)

after RFE, suggesting their limited relevance to the model. Network activity features are extracted from the Cuckoo report fields `connects_ip`, `connects_host`, and `resolves_host` (see Table 3) and encoded as binary presence–absence indicators of individual IP addresses and hostnames, yielding a high-cardinality, sparse representation. Under this granularity, most network features occur infrequently and exhibit weak association with the class label, with only 3 out of 4814 features surviving the MI threshold of 0.01 in Stage 1. Many retained features correspond to benign, environment-driven activity common to both goodware and ransomware and do not improve balanced accuracy once host-based behavioural features are included, leading to their elimination during RFE in Stage 2. The absence of network features in the final set reflects limitations in feature granularity rather than the intrinsic irrelevance of network behaviour, as the extracted network features correspond to concrete IP addresses and domains rather than aggregated or flow-level behaviour. Consequently, richer network representations may increase their discriminative value in future work.

Result 2: Impact of feature selection

The proposed two-stage feature selection method reduced over 6.4 million features to 483, significantly improving model efficiency and interoperability.
In addition, Table 7 shows how feature selection reshapes the distribution of behavioural feature groups. While `STR` and `FILE` features initially dominate the dataset, comprising over 88% of all features, recursive feature elimination reduces the prominence of `STR` (46.58%) and `FILE` to just 3.31%. However, the relative importance of `REG` and `API` features increased, reflecting their relevance to ransomware detection. In contrast, `NET` features are entirely removed, indicating limited discriminative value. Overall, the process highlights string, registry, API and system operation behaviours as the most informative.

6.3. Feature importance (RQ3)

Global feature importance refers to the overall contribution of each feature to a model's decision-making process across the entire dataset. It quantifies how much each feature influences the model's predictions, providing insights into which features are most critical for the model's behaviour. In this study, we utilised SHAP (SHapley Additive exPlanations). SHAP is a powerful and widely used method for model interpretability that provides a unified framework for explaining predictions of machine learning models. It is based on Shapley values, a concept borrowed from cooperative game theory, which assigns each feature an importance value based on its contribution to the model's output.

Figs. 10 and 11 show the SHAP bar plot and SHAP violin plot of the top 50 features that contributed the most to the model's prediction, respectively. It ranks features based on their mean absolute SHAP values, reflecting their impact on the model's predictions. Closely examining the top 10 features identified by the SHAP analysis provides key insights into how the model distinguishes ransomware from goodware. The highest-ranked feature, `(STRING:!this program cannot be run in dos mode)` with a SHAP value of +1.27, indicates that the model flags executables designed for modern systems, a common trait of ransomware. Similarly, `(API:LdrGetProcedureAddress)` follows by identifying a function call crucial to dynamic linking in Windows. Malware often relies on this technique to load external, hidden code into running processes. This feature signals the model's sensitivity to dynamic linking, a tactic frequently exploited by ransomware to inject malicious payloads. Together with the first feature, it suggests a strong reliance on identifying signs of obfuscation and covert execution. The model also prioritises `entropy-based features` such as `(SIGNATURE:packer_entropy)`, which detects packed files commonly used by ransomware to obfuscate their code.

Table 8
Feature group distribution of the top 50 features based on SHAP. The feature groups are: application programming interface (API), registry keys (REG), file operations (FILE), directory operations (DIR), strings (STR), network activities (NET), system operations (SYS), drop operations (DROP), and signatures (SIG). N represents the number of features belonging to that group and % represents the percentage of each feature group relative to the total features selected by SHAP in the top 50.

Feature groups	N	%
API	13	26.0
REG	8	16.0
FILE	1	2.0
DIR	0	0.0
STR	11	22.0
NET	0	0.0
SYS	11	22.0
DROP	2	4.0
SIG	4	8.0
Total	50	

Additionally, `(SIGNATURE:allocates_rwx)` and `(API:NtAllocateVirtualMemory)` reflect the model's ability to identify suspicious memory allocation practices typical of ransomware that manipulates system processes for stealthy execution. The feature `(SIGNATURE:allocates_rwx)` is memory allocation with Read-Write-Execute (RWX) permissions. This tactic is a hallmark of ransomware, as it allows the injection of executable code into running processes. The prominence of this feature signals the model's sharp focus on detecting suspicious memory behaviours, particularly those indicative of attempts to execute hidden or harmful code. The model continues to build this case with `(API:NtAllocateVirtualMemory)`, focusing on memory allocation within a process's address space. This API call is often associated with malware activities such as runtime payload injection and manipulation of system memory. By highlighting this feature, the model further emphasises its sensitivity to runtime behaviours commonly seen in ransomware.

As we move forward, `(STRING:kernel32.dll)` introduces an important system-level component. `Kernel32.dll` is essential for core Windows functions, and its interaction with software could signal attempts by malware to manipulate system-level operations. This feature enhances the model's ability to detect interactions with critical system resources, which are frequently targeted by malicious software seeking to gain elevated privileges or maintain persistence.

Furthermore, `(API:CoUninitialize)` shifts the focus to the use of COM (Component Object Model) components. Although not inherently malicious, COM is often exploited by malware to execute hidden functions. The model uses this feature to spot unusual interactions with COM objects, an indication that the software might be attempting to mask its true purpose, a strategy commonly adopted by ransomware.

Features like `(SYSTEM:DLL_LOADED:uxtheme.dll)` and `(API:CreateDirectoryW)` further indicate the model's detection of legitimate system behaviours hijacked by ransomware, such as loading system DLLs or creating directories for malicious payloads. These top features show that the model successfully identifies malicious activities related to **dynamic code execution**, **memory manipulation**, and **system modifications**, which are characteristic of modern ransomware attacks.

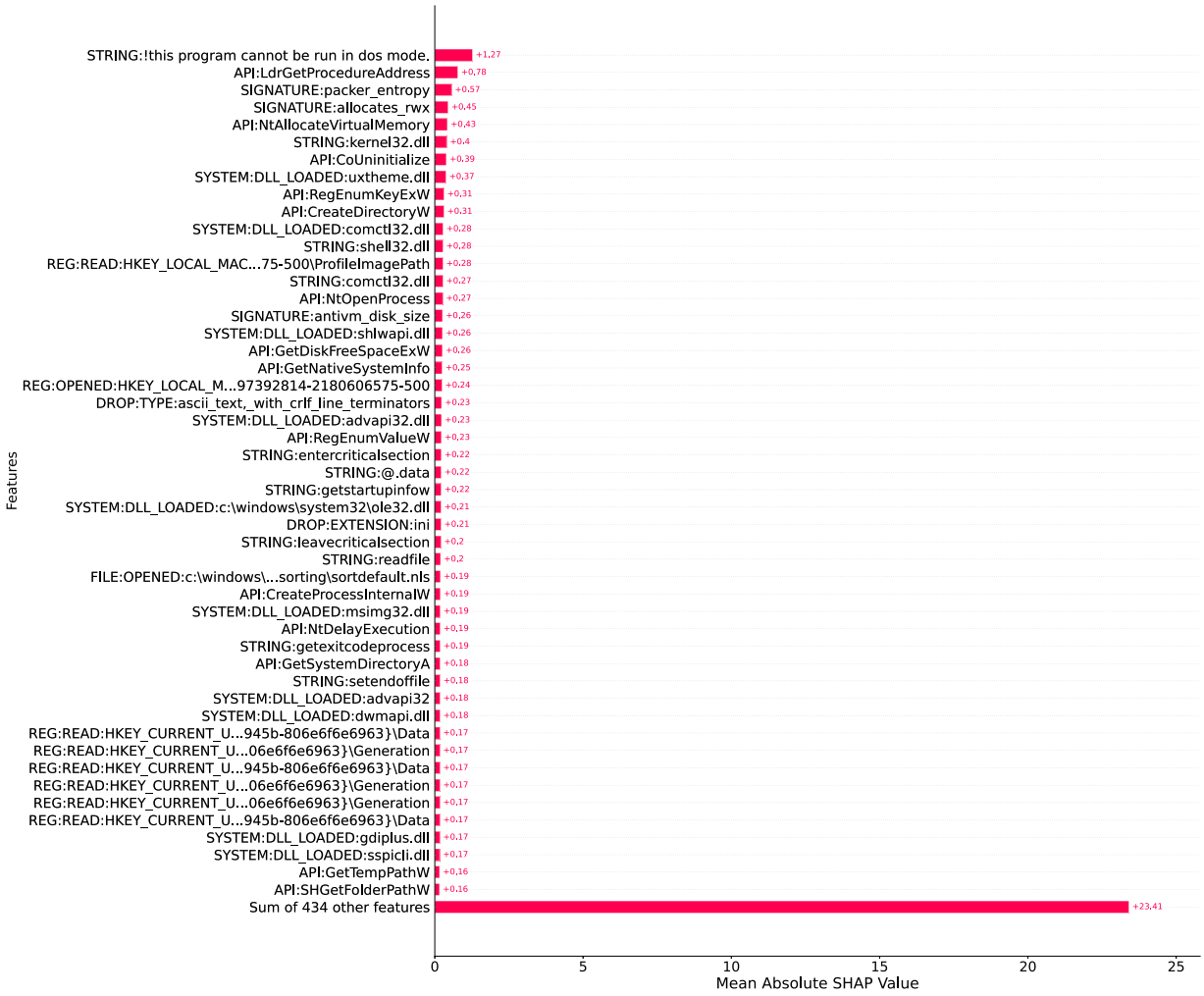


Fig. 10. SHAP decision plot showing the top 50 features ranked by their impact on model output.

Furthermore, we analyse the top 50 features based on SHAP into their respective feature groups as shown in Table 8. **API** is the highest-ranking feature group, with 13 features (26%), indicating the model's strong reliance on API calls to detect ransomware and goodware dynamic behaviours. This group is critical in identifying how software interacts with the system at runtime, providing insights into potential malicious behaviour. Following closely are the **STR** and **SYS** groups, each contributing 11 features (22%). **STR** features focus on string patterns, such as file paths or executable names, which can highlight signatures of malicious software. On the other hand, the **SYS** group detects system-level operations like memory manipulation and interaction with critical system components, which are often exploited by ransomware to gain control or elevate privileges.

The next most frequent group is **REG** with 8 features (16%), highlighting the importance of registry interactions in detecting malware persistence and system alterations. **SIG** follows with 4 features (8%), aiding in detecting known malicious software through signature-based methods. The **DROP** group contributes 2 features (4%), reflecting sensitivity to file drop operations, commonly used by ransomware.

The **FILE**, **DIR**, and **NET** groups contribute the least, with **FILE** representing 1 feature (2%) and **DIR** and **NET** having no features (0%), suggesting they play a minor role in the top 50 features.

Result 3: Most important features

SHAP analysis shows that API calls, string patterns, system behaviours, and registry keys are the most predictive feature groups, making up over 70% of the top 50 features. Key features include dynamic linking, memory use, and interactions with critical components. File, directory, and network feature groups contribute little, reflecting the model's focus on runtime and obfuscation behaviour.

6.4. Model decision analysis (RQ4)

6.4.1. Misclassified model decisions

Fig. 12 visualises the performance of the Logistic Regression model in classifying goodware and ransomware. The rows represent the true labels, while the columns represent the predicted labels. The matrix indicates that the model correctly identified 498 instances of goodware and 459 instances of ransomware. There were 12 goodware instances misclassified as ransomware (false positives) and 6 ransomware instances misclassified as goodware (false negatives), achieving an accuracy of 98.15% as shown in Table 6.

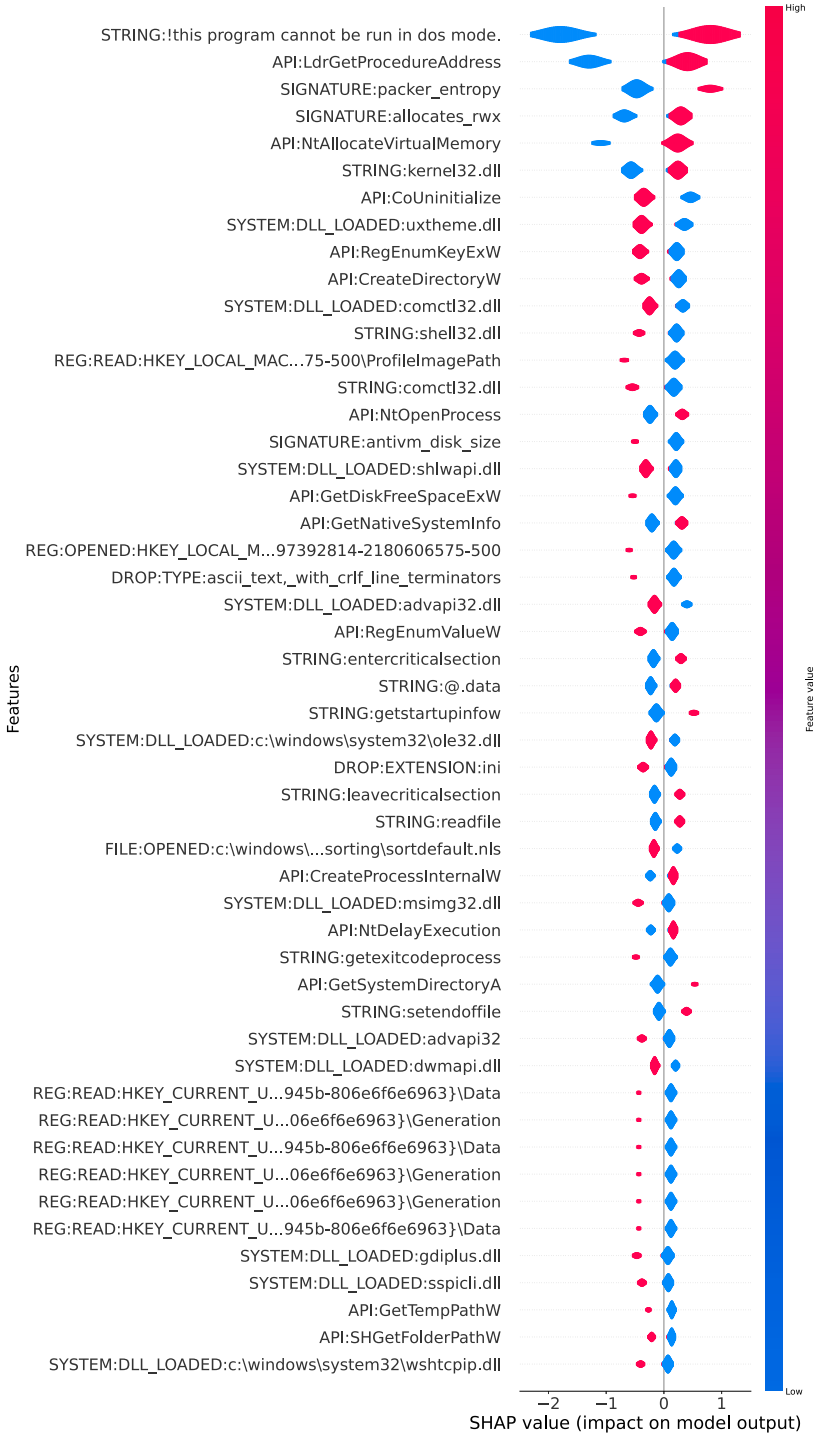


Fig. 11. SHAP violin plot showing the top 50 features ranked by their impact on model output. Wider sections indicate features with greater and more variable influence, while positive and negative values reflect their contribution to the positive and negative class predictions, respectively.

Out of the 6 ransomware instances misclassified as goodware, the misclassifications were distributed across three ransomware families: **Shodi** (4 instances), **Clop** (1 instance), and **Delshad** (1 instance). These misclassifications reflect the model's difficulty in distinguishing between these ransomware families. The misclassified instances also span two ransomware types, including **Crypto** (5 instances) and **Modern** (1 instance). Also, the misclassified ransomware instances span across different **first submission years**, with 4 instances first seen in 2021, 1 instance in 2023, and 1 instance in 2024.

The 12 **goodware** samples misclassified as **ransomware** are associated with several **software sources**, with the majority (10 instances) originating from *Software Informer's Most Popular* category, and the remaining instances from *Education* (1), and *Business* (1). These misclassified samples include legitimate software programs such as `pdfcreator`, `zoominstallerfull`, `teamspeak3-client`, and `avast_secure_browser_setup`, among others.

The misclassified samples are also distributed across different **first submission years**: 2024 (7 instances), 2023 (4 instances), and 2022 (1

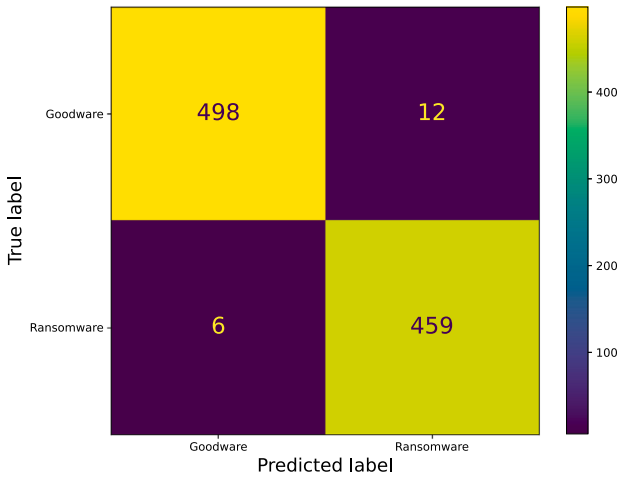


Fig. 12. Confusion matrix for the ransomware detection logistic regression model based on the two-stage feature selection technique. The results show that the model correctly classified 498 Goodware and 459 Ransomware instances, with 12 false positives and 6 false negatives.

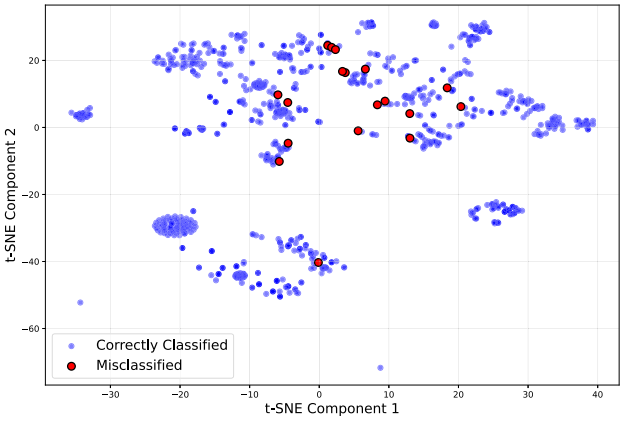


Fig. 13. t-SNE projection in 2D with misclassified samples highlighted in red with black borders and correctly classified in blue.

instances). This suggests that the model is generally more effective at classifying goodware from earlier years (2022) but struggles more with programs submitted in the last two years (2023 and 2024). The model might encounter challenges due to new software or updates that exhibit behaviour similar to ransomware, leading to false positives, especially for popular software versions.

To investigate the decision boundary and understand where the misclassified samples fall in the feature space, we used the dimensionality reduction technique, t-Distributed Stochastic Neighbour Embedding (t-SNE) to project the high-dimensional data into a 2D and 3D space for visualisation as shown in Figs. 13 and 14 respectively. Misclassified samples are highlighted in red with black borders, and correctly classified samples are in blue. As observed in both plots, the correctly classified samples tend to form distinct clusters, indicating that the model can effectively separate different classes in the feature space. The misclassified samples are scattered near the correctly classified ones, suggesting that the model struggles to differentiate between certain samples. These samples might be ambiguous or fall near the decision boundary of the model.

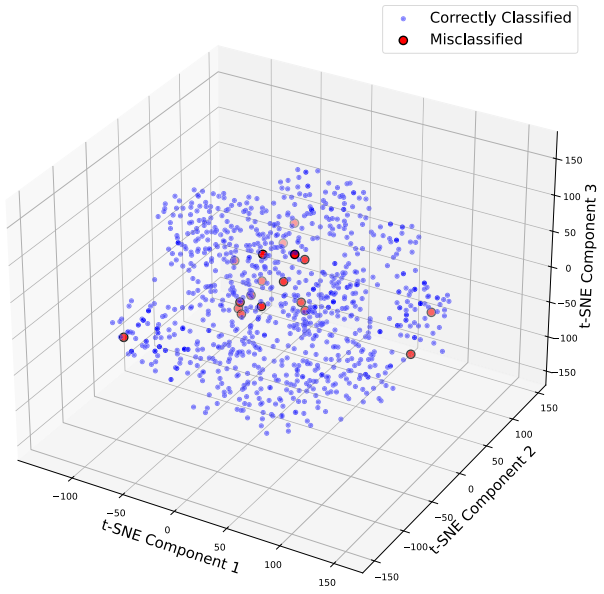


Fig. 14. t-SNE projection in 3D with misclassified samples highlighted in red with black borders and correctly classified in blue.

Result 4: Misclassified model decision analysis

Misclassification analysis reveals that most false positives involve recent, popular goodware exhibiting ransomware-like behaviour, while false negatives cluster around a few ransomware families. t-SNE visualisations show misclassified samples near decision boundaries, indicating ambiguity. These insights expose model weaknesses and inform strategies to improve robustness and interpretability.

6.4.2. Local misclassified instance explanation

To understand why the model misclassified certain instances, we conducted a local instance analysis using Local Interpretable Model-agnostic Explanations (LIME). This method helps to clarify the reasons behind the model's specific predictions.

Fig. 15 shows the LIME explanation for why a ransomware sample was misclassified as goodware. This misclassification is primarily due to the model's reliance on features typically associated with goodware. Specifically, `Feature (SIGNATURE:packer_entropy)` strongly favoured goodware, contributing significantly to the probability of classifying the sample as goodware. While `Feature (STRING:!this program cannot be run in dos mode)` and `Feature (REG:READ:HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\ProfileList\S-1-5-21-4114181432-2397392814-2180606575-500\ProfileImagePath)` indicated ransomware. The model's emphasis on goodware-associated features outweighed the signals from ransomware-associated features. This highlights the challenge of distinguishing between the two classes when feature values overlap or when certain features disproportionately influence the model's decision-making process.

Similarly, Fig. 16 shows the LIME explanation of a goodware sample misclassified as ransomware. It shows that the model was heavily influenced by `Feature (STRING:!this program cannot be run in dos mode)`, with a probability of `0.26`, which is associated with Ransomware. Despite other features like `Feature (SIGNATURE:packer_entropy)` and `Feature (SIGNATURE:allocates_rwx)` showing benign behaviour, the model's reliance on the strong ransomware-associated features led to the misclassification.

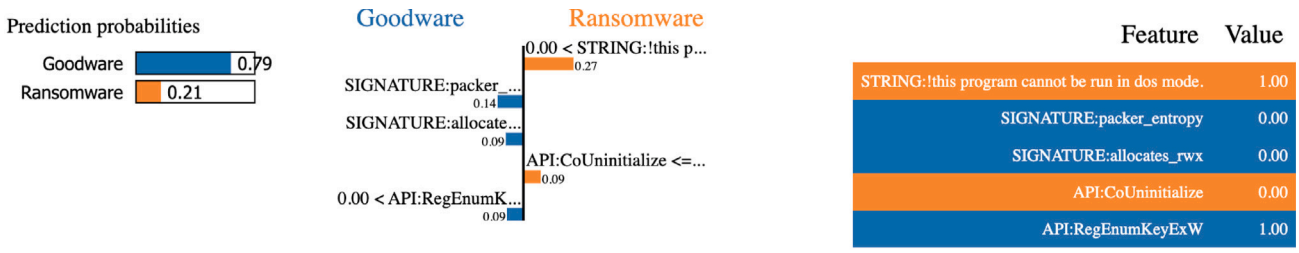


Fig. 15. LIME explanation of the model's prediction for a ransomware sample misclassified as goodware. The figure displays the top 5 features and the predicted probabilities for Goodware (79%) and Ransomware (21%), along with the contributions of individual features to the model's decision. Features influencing the Goodware prediction are shown on the left, while those contributing to the Ransomware classification are shown on the right. The feature values for the misclassified sample are also presented.



Fig. 16. LIME explanation of the model's prediction for a goodware sample misclassified as ransomware. The figure displays the top 5 features and the predicted probabilities for Goodware (13%) and Ransomware (87%), along with the contributions of individual features to the model's decision. Features influencing the Goodware prediction are shown on the left, while those contributing to the Ransomware classification are shown on the right. The feature values for the misclassified sample are also presented.

Result 5: Local misclassified instance explanation

LIME analysis reveals misclassifications often occur when the model overweights a few dominant features. In ransomware samples misclassified as goodware, benign indicators like low entropy suppressed the influence of malicious signals. Conversely, goodware samples were misclassified as ransomware due to strong associations with features commonly seen in malware, such as specific string patterns and registry accesses.

7. Discussion

This paper has presented a novel approach to ransomware detection using machine learning. To contextualise our findings and provide a broader understanding of the field, this section critically reviews existing literature on ransomware detection using machine learning, focusing on the efficacy of traditional classifiers, the role of feature selection, the predictive power of different feature groups, and the insights gained from analysing misclassified instances. Also, we discuss the threat to validity and limitations of this study.

7.1. Machine learning models for ransomware detection

7.1.1. Ransomware-goodware classification

Our results support prior findings that traditional machine learning classifiers, particularly logistic regression, are highly effective for ransomware-goodware classification when trained on behavioural features. Similar to the EldeRan dataset (Sgandurra et al., 2016), our logistic regression model achieved strong performance, but with the added benefit of enhanced efficiency through our proposed two-stage feature selection technique. This approach significantly reduced the feature space, from over 6 million to just 483, while maintaining high accuracy, offering a practical and interpretable solution for real-world deployment.

Contrary to earlier studies that favour ensemble methods for ransomware detection (Jaya and Razak, 2022; Aljabri et al., 2024), our evaluation showed that LightGBM performed worst, with lower accuracy and substantially higher computation time. This suggests that simpler models can outperform more complex alternatives when paired with well-chosen features, especially on high-dimensional behavioural data. Moreover, while most previous work focuses solely on predictive performance, our study emphasises the importance of model efficiency and feature reduction, addressing a key limitation in the current literature. Overall, our findings highlight that logistic regression, combined with lightweight feature selection, offers a robust and scalable approach to binary ransomware detection.

While the proposed approach achieves strong performance for ransomware detection and type-level classification, performance at the family level is comparatively lower and provides opportunity for improvement. Nevertheless, the achieved performance remains well above random chance for a 65-class classification problem, indicating that meaningful behavioural signals are captured. The observed gap highlights the increased difficulty of fine-grained family discrimination and suggests that family-level classification represents the primary opportunity for further improvement, potentially through more expressive representations or modelling approaches.

7.1.2. Impact of feature selection

Effective feature selection is crucial in dynamic ransomware detection due to the high dimensionality of behavioural data. Our two-stage approach (applying MI with a 0.01 threshold followed by RFE) reduced the MLRan dataset from over 6.4 million to just 483 features, significantly improving efficiency and interpretability without sacrificing performance. This builds on prior work using MI (Sgandurra et al., 2016; Abbasi et al., 2020), but improves upon it by avoiding fixed feature quotas instead refining selection through performance-guided elimination.

While studies such as Moreira et al. (2022) and Onwuegbuche et al. (2023) have shown the benefits of reducing feature sets to 300–500 using various techniques, our method achieves similar reductions from a much larger initial space, demonstrating greater scalability. Importantly, our results confirm that the most predictive behaviours lie in API calls, registry, strings, and system operations, echoing findings

from previous work. Overall, our approach offers a simple yet powerful framework for reducing noise, improving model generalisation, and capturing core ransomware behaviours in large-scale datasets.

7.1.3. Important features

Our SHAP analysis highlights API calls, string patterns, system behaviours, and registry activity as the most predictive features for ransomware detection, accounting for over 70% of the top 50 features. This complements findings by Gulmez et al. (2024), who also identified API and registry features as strong ransomware indicators, though they found mutexes and DLLs more relevant in some contexts. Instead of using focused input types, our broader feature set reveals that runtime behaviours, such as dynamic linking and memory allocation, are key discriminators. Overall, our results reinforce that models benefit most from features capturing obfuscation, memory manipulation, and system-level interactions, and demonstrate the value of SHAP for uncovering these behavioural signals.

7.1.4. Model decision analysis

Understanding why models misclassify samples is critical for improving detection robustness. Our LIME-based analysis shows that misclassifications often stem from the model's overreliance on a few dominant features. In false negatives, benign indicators such as low entropy masked ransomware signals, while false positives occurred when benign software exhibited traits commonly associated with malware, like specific strings or registry activity. These findings align with Gulmez et al. (2024), who observed that cryptographic APIs and ambiguous DLLs contributed to misclassification, highlighting the nuanced interplay of features rather than their individual presence. However, our work extends prior efforts by demonstrating how local explanations reveal imbalances in feature influence, even within correctly engineered models. By exposing instances where legitimate behaviours are misinterpreted due to overlapping traits with ransomware, our analysis underscores the need for more discriminative and context-aware features.

7.2. Threat to validity

While this study takes deliberate steps to ensure methodological soundness in constructing and evaluating the MLRan dataset, we acknowledge potential threats to validity that, while mitigated, cannot be entirely eliminated.

7.2.1. Construct validity

MLRan is built using behavioural data collected from dynamic analysis in sandboxed environments, which may not perfectly replicate all real-world execution conditions. However, we addressed this by incorporating a diverse set of ransomware and goodware samples and ensuring the sandboxes closely replicate a real-world system. Furthermore, careful labelling and validation steps were taken to reduce misclassification risk, though the evolving nature of malware may still introduce edge cases that challenge labelling frameworks.

7.2.2. Internal validity

Using mutual information followed by recursive feature elimination, the two-stage feature selection approach was designed to enhance model interpretability and reduce overfitting. Although thresholds and model parameters were chosen based on empirical performance and prior literature, we recognise that some subtle features may be filtered out. To address this, we validated performance across multiple models and feature groups, ensuring that the retained features remained representative of key ransomware behaviours.

7.2.3. External validity

While our models and findings are based on the MLRan dataset, which captures a rich and diverse set of ransomware and goodware behaviours, generalisability to unseen environments or malware families may vary. We mitigated this by including recent and varied ransomware types and conducting binary and multiclass evaluations. Nevertheless, further validation on independent datasets would help extend the applicability of the findings.

7.2.4. Interpretability validity

We employed state-of-the-art XAI methods, including SHAP and LIME, to provide insights into model decision-making. Although these techniques offer robust explanations, their assumptions (e.g., feature independence in SHAP, local approximation in LIME) may not fully capture all aspects of model logic. To address this threat to validity, we used global and local explanation methods and cross-referenced feature importance rankings with domain knowledge to ensure interpretative reliability.

In summary, while no empirical study is entirely free from validity concerns, we took rigorous and transparent steps to reduce their impact. The design of MLRan, the careful selection and validation of features, and the application of interpretable machine learning all contribute to the reliability and reproducibility of the presented results.

8. Conclusion and future work

This study presents a comprehensive framework for advancing behavioural ransomware detection by creating the MLRan ransomware dataset (the largest and most diverse publicly available behavioural ransomware dataset to date) and developing practical open-source tools for advancing ransomware research. We introduced GUIDE-MLRan, a set of structured guidelines to support reproducible, high-quality ransomware dataset construction. We applied these principles to develop MLRan. Covering four major ransomware types and 64 families, and spanning nine behavioural feature groups, MLRan fills a critical gap in existing datasets by offering breadth, depth, and balance.

To facilitate scalable data collection, we enhanced Cuckoo Sandbox through automation scripts for file submission and result sorting, reducing manual effort and improving consistency. Using this infrastructure, we constructed a robust machine learning pipeline that integrates a novel two-stage feature selection method (mutual information followed by recursive feature elimination), which reduced the feature space from over 6.4 million to 483 while maintaining high classification performance. This reduction significantly improves model training time and interpretability. Through global (SHAP) and local (LIME) explainable AI techniques, we further analysed the models' decision-making, identifying API usage, string patterns, memory operations, and registry behaviours as the most predictive signals of ransomware activity. These findings validate the effectiveness of MLRan and contribute to a deeper understanding of ransomware behaviour. While our results demonstrate strong model performance in binary classification, they also reveal challenges in multiclass settings and instances of feature ambiguity leading to misclassification. This underscores the need for continued refinement of feature engineering and interpretability tools.

In future work, we will continue our research following several directions. First, we will evaluate model robustness by incorporating adversarial samples in the dataset to study model robustness against evasion techniques. Second, we will develop context-aware models that consider temporal sequences and inter-feature dependencies using graph learning or attention-based architectures. In parallel, we will leverage the temporal span of the dataset to conduct longitudinal analyses of ransomware evolution, examining model performance degradation under concept drift across time periods and investigating retraining strategies required for sustained deployment effectiveness. Third, to improve the classification of ransomware types and families, future work will investigate alternative modelling approaches, including deep learning and graph-based architectures such as graph neural networks over API call sequences or behavioural graphs, to better capture complex behavioural dependencies. Fourth, while this

Table 9
Model hyperparameters and justifications for each algorithm used in the study, including their value ranges and rationale based on empirical or theoretical considerations.

Model	Hyperparameter	Values	Justification
LightGBM	num_leaves	[15, 31, 63]	Controls model complexity; powers of 2 allow a trade-off between expressiveness and overfitting.
	n_estimators	[50, 100, 200]	More trees improve stability and performance at the cost of training time.
	learning_rate	[0.01, 0.1]	Smaller values converge more smoothly and generalise better.
Decision Tree	max_depth	[None, 10, 20]	Restricts depth to prevent overfitting; None allows full growth.
	min_samples_split	[2, 5, 10]	Higher values reduce overfitting by requiring more samples per split.
Logistic Regression	C	[0.01, 0.1, 1, 10]	Inverse regularisation strength; wider range allows control over model complexity.
	penalty	['l2']	Standard regularisation for numerical stability and generalisation.
	solver	['lbfgs' (multiclass), 'liblinear' (binary)]	Solvers selected for efficiency and compatibility with classification type.
Random Forest	n_estimators	[50, 100, 200]	Larger ensembles reduce variance; diminishing returns beyond a point.
	max_depth	[None, 10, 20]	Controls overfitting; allows shallower trees for better generalisation.
	min_samples_split	[2, 5]	Prevents overly specific branches that may not generalise well.
Extra Trees	n_estimators	[50, 100, 200]	Similar to Random Forest; more trees enhance robustness.
	max_depth	[None, 10, 20]	Regulates tree complexity and training time.
	min_samples_split	[2, 5]	Enforces minimum data per split; helps mitigate overfitting.

study focuses on Windows PE ransomware due to its prevalence, the proposed guidelines (GUIDE-MLRan), dataset construction pipeline, and behavioural feature extraction methods are platform-agnostic and can be extended to other operating systems, including Android, Linux, and macOS, by adopting appropriate sandboxing environments and OS-specific behavioural artefacts. Finally, we encourage the community to use, extend, and benchmark on MLRan, as all datasets, tools, and code are openly released to foster reproducibility and collaboration in this critical domain.

CRediT authorship contribution statement

Faithful Chiagoziem Onwuegbuche: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Sunday Olaoluwa Adelodun:** Software, Investigation. **Anca Delia Jurcut:** Writing – review & editing, Supervision, Methodology, Conceptualization. **Liliana Pasquale:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was funded by Research Ireland through the Research Ireland Centre for Research Training in Machine Learning (18/CRT/6183).

This article was supported in part by Research Ireland Grant 13/RC/2094_2. We also acknowledge Răzvan Maioru for his contribution in analysing a few of the samples using the Cuckoo sandbox.

Appendix

See Table 9.

Data availability

The MLRan dataset, along with all associated code and tools, is publicly available at: https://github.com/faithfulco/mlran.

References

Abbasi, M.S., Al-Sahaf, H., Welch, I., 2020. Particle swarm optimization: A wrapper-based feature selection method for ransomware detection and classification. In: Applications of Evolutionary Computation: 23rd European Conference, EvoApplications 2020, Held As Part of EvoStar 2020, Seville, Spain, April 15–17, 2020, Proceedings 23. Springer, pp. 181–196.

Ahmed, M., 2024. Ransomware Evolution. CRC Press.

Al-Hawawreh, M., Alazab, M., Ferrag, M.A., Hossain, M.S., 2024. Securing the industrial internet of things against ransomware attacks: A comprehensive analysis of the emerging threat landscape and detection mechanisms. J. Netw. Comput. Appl. 223, 103809.

Aljabri, M., Alhaidari, F., Albuainain, A., Alrashidi, S., Alansari, J., Alqahtani, W., Alshaya, J., 2024. Ransomware detection based on machine learning using memory features. Egypt. Inform. J. 25, 100445.

Allix, K., Bissyandé, T.F., Klein, J., Le Traon, Y., 2015. Are your training datasets yet relevant? an investigation into the importance of timeline in machine learning-based malware detection. In: International Symposium on Engineering Secure Software and Systems. Springer, pp. 51–67.

Alraizza, A., Algarni, A., 2023. Ransomware detection using machine learning: A survey. Big Data Cogn. Comput. 7 (3), 143.

Alrawi, O., Wong, M.Y., Avgetidis, A., Valakuzhy, K., Adjibi, B.V., Karakatsanis, K., Ahamad, M., Blough, D., Monrose, F., Antonakakis, M., 2024. SoK: An essential guide for using malware sandboxes in security applications: Challenges, pitfalls, and lessons learned. arXiv preprint arXiv:2403.16304.

Anand, C., Shanker, R., 2023. Advancing crypto ransomware with multi level extortion: A peril to critical infrastructure. In: 2023 2nd International Conference for Innovation in Technology. INOCON, IEEE, pp. 1–5.

Andronio, N., Zanero, S., Maggi, F., 2015. Heldroid: Dissecting and detecting mobile ransomware. In: Research in Attacks, Intrusions, and Defenses: 18th International Symposium, RAID 2015, Kyoto, Japan, November 2-4, 2015. Proceedings 18. Springer, pp. 382–404.

Apruzzese, G., Laskov, P., Tastemirova, A., 2022. SoK: The impact of unlabelled data in cyberthreat detection. In: 2022 IEEE 7th European Symposium on Security and Privacy. EuroS&P, IEEE, pp. 20–42.

Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K., 2022. Dos and don'ts of machine learning in computer security. In: 31st USENIX Security Symposium. USENIX Security 22, pp. 3971–3988.

Ashraf, A., Aziz, A., Zahoora, U., Rajarajan, M., Khan, A., 2019. Ransomware analysis using feature engineering and deep neural networks. arXiv preprint arXiv:1910.00286.

Baig, Z., Mekala, S.H., Zeadally, S., 2023. Ransomware attacks of the COVID-19 pandemic: Novel strains, victims, and threat actors. IT Prof. 25 (5), 37–44.

Beaman, C., Barkworth, A., Akande, T.D., Hakak, S., Khan, M.K., 2021. Ransomware: Recent advances, analysis, challenges and future research directions. Comput. Secur. 111, 102490.

Bengio, Y., Goodfellow, I., Courville, A., 2017. Deep Learning. vol. 1, MIT press Cambridge, MA, USA.

Botacin, M., Ceschin, F., Sun, R., Oliveira, D., Grégio, A., 2021. Challenges and pitfalls in malware research. Comput. Secur. 106, 102287.

Cai, L., Zhu, Y., 2015. The challenges of data quality and data quality assessment in the big data era. Data Sci. J. 14, 2–2.

Ceschin, F., Botacin, M., Bifet, A., Pfahringer, B., Oliveira, L.S., Gomes, H.M., Grégio, A., 2024. Machine learning (in) security: A stream of problems. Digit. Threat.: Res. Pr. 5 (1), 1–32.

Chen, H., Chen, J., Ding, J., 2021. Data evaluation and enhancement for quality improvement of machine learning. IEEE Trans. Reliab. 70 (2), 831–847.

Chio, C., Freeman, D., 2018. Machine Learning and Security: Protecting Systems with Data and Algorithms. " O'Reilly Media, Inc.".

Continella, A., Guagnelli, A., Zingaro, G., De Pasquale, G., Barenghi, A., Zanero, S., Maggi, F., 2016. Shieldfs: a self-healing, ransomware-aware filesystem. In: Proceedings of the 32nd Annual Conference on Computer Security Applications. pp. 336–347.

Cortes, C., Mohri, M., Riley, M., Rostamizadeh, A., 2008. Sample selection bias correction theory. In: International Conference on Algorithmic Learning Theory. Springer, pp. 38–53.

CrowdStrike, 2025. State of ransomware survey 2025. (Accessed 12 January 2026), https://www.crowdstrike.com/en-us/resources/reports/state-of-ransomware-survey/.

Eren, M.E., Bhattarai, M., Joyce, R.J., Raff, E., Nicholas, C., Alexandrov, B.S., 2023. Semi-supervised classification of malware families under extreme class imbalance via hierarchical non-negative matrix factorization with automatic model selection. ACM Trans. Priv. Secur. 26 (4), 1–27.

Fernando, D.W., Komninos, N., 2024. Fesad ransomware detection framework with machine learning using adaption to concept drift. Comput. Secur. 137, 103629.

Fisher, T., Pieri, Z., Howell, C.J., O'Malley, R., Tremblay, L., Dawoud, M., 2025. Vendor communication themes in darknet ransomware-as-a-service (raas) advertisements. Comput. Hum. Behav. 165, 108571.

Gao, H., Zahedi, M., Treude, C., Rosenstock, S., Cheong, M., 2024. Documenting ethical considerations in open source ai models. In: Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement. pp. 177–188.

Gong, Y., Liu, G., Xue, Y., Li, R., Meng, L., 2023. A survey on dataset quality in machine learning. Inf. Softw. Technol. 162, 107268.

Gulmez, S., Kakisim, A.G., Sogukpinar, I., 2024. XRan: Explainable deep learning-based ransomware detection using dynamic analysis. Comput. Secur. 139, 103703.

Hastie, T., Tibshirani, R., Friedman, J., 2017. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Springer.

He, H., Garcia, E.A., 2009. Learning from imbalanced data. IEEE Trans. Knowl. Data Eng. 21 (9), 1263–1284.

Herrera-Silva, J.A., Hernández-Álvarez, M., 2023. Dynamic feature dataset for ransomware detection using machine learning algorithms. Sensors 23 (3), 1053.

Hirano, M., Hodota, R., Kobayashi, R., 2022. RanSAP: An open dataset of ransomware storage access patterns for training machine learning models. Forensic Sci. Int.: Digit. Investig. 40, 301314.

Hirano, M., Kobayashi, R., 2025. RanSMAP: Open dataset of ransomware storage and memory access patterns for creating deep learning based ransomware detectors. Comput. Secur. 150, 104202.

Holzinger, A., Kieseberg, P., Tjoa, M., Weippl, E., 2019. Machine learning and knowledge extraction.

Homayoun, S., Dehghantanha, A., Ahmadzadeh, M., Hashemi, S., Khayami, R., 2017. Know abnormal, find evil: frequent pattern mining for ransomware threat hunting and intelligence. IEEE Trans. Emerg. Top. Comput. 8 (2), 341–351.

Hou, Y., Guo, L., Zhou, C., Xu, Y., Yin, Z., Li, S., Sun, C., Jiang, Y., 2024. An empirical study of data disruption by ransomware attacks.

Hurier, M., Suarez-Tangil, G., Dash, S.K., Bissyandé, T.F., Le Traon, Y., Klein, J., Cavallaro, L., 2017. Euphony: Harmonious unification of cacophonous anti-virus vendor labels for android malware. In: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories. MSR, IEEE, pp. 425–435.

Ispahany, J., Islam, M.R., Islam, M.Z., Khan, M.A., 2024. Ransomware detection using machine learning: A review, research limitations and future directions. IEEE Access.

Jaya, M.I., Razak, M.F.A., 2022. Dynamic ransomware detection for windows platform using machine learning classifiers. JOIV: Int. J. Inform. Vis. 6 (2–2), 469–474.

Jeremiah, S.R., Chen, H., Gritzalis, S., Park, J.H., 2024. Leveraging application permissions and network traffic attributes for android ransomware detection. J. Netw. Comput. Appl. 230, 103950.

Jethva, B., Traoré, I., Ghaleb, A., Ganame, K., Ahmed, S., 2020. Multilayer ransomware detection using grouped registry key operations, file entropy and file signature monitoring. J. Comput. Secur. 28 (3), 337–373.

Jiang, Y., Li, G., Li, S., Guo, Y., 2024. BenchMFC: A benchmark dataset for trustworthy malware family classification under concept drift. Comput. Secur. 139, 103706.

Joyce, R.J., Amlani, D., Nicholas, C., Raff, E., 2023. Motif: A malware reference dataset with ground truth family labels. Comput. Secur. 124, 102921.

Kamolov, S., 2021. Feature selection: state-of-the-art survey. Ann. Math. Comput. Sci. 4, 48–54.

Kang, M., Tian, J., 2018. Machine learning: Data pre-processing. In: Prognostics and Health Management of Electronics: Fundamentals, Machine Learning, and the Internet of Things. Wiley Online Library, pp. 111–130.

Kapoor, A., Gupta, A., Gupta, R., Tanwar, S., Sharma, G., Davidson, I.E., 2021. Ransomware detection, avoidance, and mitigation scheme: A review and future directions. Sustainability 14 (1), 8.

KELA, 2025. Escalating ransomware threats to national infrastructure. (Accessed 12 January 2026), https://www.kelacyber.com/resources/research/escalating-ransomware-threats-to-national-security/.

Kelleher, J.D., Mac Namee, B., D'arcy, A., 2020. Fundamentals of Machine Learning for Predictive Data Analytics: Algorithms, Worked Examples, and Case Studies. MIT Press.

Kelly, R., 2025. Ransomware victims are refusing to play ball with hackers: Just 17% of enterprises have paid up so far in 2025, marking an all-time low. Published: 15 August 2025; (Accessed 12 January 2026), https://www.itpro.com/business/business-strategy/ransomware-victims-are-refusing-to-play-ball-with-hackers-just-17-percent-of-enterprises-have-paid-up-so-far-in-2025-marking-an-all-time-low.

Kerns, Q., Payne, B., Abegaz, T., 2022. Double-extortion ransomware: A technical analysis of maze ransomware. In: Proceedings of the Future Technologies Conference (FTC) 2021, Volume 3. Springer, pp. 82–94.

Küchler, A., Mantovani, A., Han, Y., Bilge, L., Balzarotti, D., 2021. Does every second count? time-based evolution of malware behavior in sandboxes. In: NDSS 2021, Network and Distributed Systems Security Symposium. Internet Society.

Leguesse, Y., Vella, M., Ellul, J., 2018. Androneo: Hardening android malware sandboxes by predicting evasion heuristics. In: Information Security Theory and Practice: 11th IFIP WG 11.2 International Conference, WISTP 2017, Heraklion, Crete, Greece, September 28–29, 2017, Proceedings 11. Springer, pp. 140–152.

Liu, K., Lu, S., Liu, C., 2014. Poster: Fingerprinting the publicly available sandboxes. In: Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security. pp. 1469–1471.

Marcinkowski, B., Goschorska, M., Wileńska, N., Siuta, J., Kajdanowicz, T., 2024. MIRAD: A method for interpretable ransomware attack detection. IEEE Access.

McIntosh, T., Kayes, A., Chen, Y.-P.P., Ng, A., Watters, P., 2021. Ransomware mitigation in the modern era: A comprehensive review, research challenges, and future directions. ACM Comput. Surv. 54 (9), 1–36.

Minnaar, A., Herbig, F.J., 2021. Cyberattacks and the cybercrime threat of ransomware to hospitals and healthcare services during the COVID-19 pandemic. Acta Criminol.: Afr. J. Criminol. Vict. 34 (3), 155–185.

Miranda, T.C., Gimenez, P.-F., Lalande, J.-F., Tong, V.V.T., Wilke, P., 2022. Debiasing android malware datasets: How can i trust your results if your dataset is biased? IEEE Trans. Inf. Forensics Secur. 17, 2182–2197.

Moreira, C.C., de Sales Jr., C.d., Moreira, D.C., 2022. Understanding ransomware actions through behavioral feature analysis. J. Commun. Inf. Syst. 37 (1), 61–76.

Moreira, C.C., Moreira, D.C., Sales Jr., C., 2024. A comprehensive analysis combining structural features for detection of new ransomware families. J. Inf. Secur. Appl. 81, 103716.

Nasir, M.S., Khan, H., Qureshi, A., Rafiq, A., Rasheed, T., 2024. Ethical aspects in cyber security maintaining data integrity and protection: A review. Spectr. Eng. Sci. 2 (3), 420–454.

National Cyber Security Centre, 2018. The Cyber Threat to UK Businesses. UK National Cyber Security Centre.

Office of Intelligence and Analysis, U.S. Department of Homeland Security, 2025. Homeland threat assessment report 2025. (Accessed 2025), https://www.dhs.gov/sites/default/files/2024-10/24_0930_ia_24-320-ia-publication-2025-hta-final-30sep24-508.pdf.

Onwuegbuche, F.C., Jurcut, A.D., Pasquale, L., 2023. Enhancing ransomware classification with multi-stage feature selection and data imbalance correction. In: International Symposium on Cyber Security, Cryptology, and Machine Learning. Springer, pp. 285–295.
Or-Meir, O., Nissim, N., Elovici, Y., Rokach, L., 2019. Dynamic malware analysis in the modern era—A state of the art survey. ACM Comput. Surv. 52 (5), 1–48.
Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., Cavallaro, L., 2019. {TesseraCT}: Eliminating experimental bias in malware classification across space and time. In: 28th USENIX Security Symposium. USENIX Security 19, pp. 729–746.
Praveen, M., Almobaideen, W., 2023. The current state of research on malware written in the rust programming language. In: 2023 International Conference on Information Technology. ICIT, IEEE, pp. 266–270.
Rahman, M.S., Coull, S., Wright, M., 2022. On the limitations of continual learning for malware classification. In: Conference on Lifelong Learning Agents. PMLR, pp. 564–582.
Raj, A., Narayan, V., Muskan, V., Sani, A., Sharma, P., Sarma, S., 2024. Modern ransomware: Evolution, methodology, attack model, prevention and mitigation using multi-tiered approach. Secur. Priv. 7 (6), e436.
Rossow, C., Dietrich, C.J., Grier, C., Kreibich, C., Paxson, V., Pohlmann, N., Bos, H., Van Steen, M., 2012. Prudent practices for designing malware experiments: Status quo and outlook. In: 2012 IEEE Symposium on Security and Privacy. IEEE, pp. 65–79.
Schwabe, D., Becker, K., Seyferth, M., Klaß, A., Schaeffter, T., 2024. The METRIC-framework for assessing data quality for trustworthy AI in medicine: a systematic review. NPJ Digit. Med. 7 (1), 203.
Sebastián, M., Rivera, R., Kotzias, P., Caballero, J., 2016. Avclass: A tool for massive malware labeling. In: Research in Attacks, Intrusions, and Defenses: 19th International Symposium, RAID 2016, Paris, France, September 19-21, 2016, Proceedings 19. Springer, pp. 230–253.
Sgandurra, D., Muñoz-González, L., Mohsen, R., Lupu, E.C., 2016. Automated dynamic analysis of ransomware: Benefits, limitations and use for detection. arXiv preprint arXiv:1609.03020.
Shiravi, A., Shiravi, H., Tavallaee, M., Ghorbani, A.A., 2012. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. Comput. Secur. 31 (3), 357–374.
Sophos, 2025. The state of ransomware 2025. (Accessed 12 January 2026), https://www.espria.com/wp-content/uploads/2025/07/sophos-state-of-ransomware-2025-1-1.pdf.
Theng, D., Bhoyar, K.K., 2024. Feature selection techniques for machine learning: a survey of more than two decades of research. Knowl. Inf. Syst. 66 (3), 1575–1637.
Thomas, D.R., Pastrana, S., Hutchings, A., Clayton, R., Beresford, A.R., 2017. Ethical issues in research using datasets of illicit origin. In: Proceedings of the 2017 Internet Measurement Conference. pp. 445–462.
Tran, N., Chen, H., Bhuyan, J., Ding, J., 2022. Data curation and quality evaluation for machine learning-based cyber intrusion detection. IEEE Access 10, 121900–121923.
Urooj, U., Al-rimy, B.A.S., Zainal, A., Ghaleb, F.A., Rassam, M.A., 2021. Ransomware detection using the dynamic analysis and machine learning: A survey and research directions. Appl. Sci. 12 (1), 172.
Verma, R.M., Zeng, V., Faridi, H., 2019. Data quality for security challenges: Case studies of phishing, malware and intrusion detection datasets. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 2605–2607.
Wand, Y., Wang, R.Y., 1996. Anchoring data quality dimensions in ontological foundations. Commun. ACM 39 (11), 86–95.
Wang, R.Y., Strong, D.M., 1996. Beyond accuracy: What data quality means to data consumers. J. Manage. Inf. Syst. 12 (4), 5–33.
Willems, C., Holz, T., Freiling, F., 2007. Toward automated dynamic malware analysis using cwsandbox. IEEE Secur. Priv. 5 (2), 32–39.
Yaseen, Q.M., 2023. The effect of the ransomware dataset age on the detection accuracy of machine learning models. Information 14 (3), 193.
Zaki, A., 2023. 85% of cybersecurity leaders say recent attacks powered by AI: Weekly stat. Published: 30 August 2023; (Accessed 12 January 2026), https://www.cfo.com/news/cybersecurity-attacks-generative-ai-security-ransom/692176/.
Zebari, R., Abdulazeez, A., Zeebaree, D., Zebari, D., Saeed, J., 2020. A comprehensive review of dimensionality reduction techniques for feature selection and feature extraction. J. Appl. Sci. Technol. Trends 1 (1), 56–70.
Zhou, Y., Cheng, G., Chen, Z., Yu, S., 2023. MalPurifier: Enhancing android malware detection with adversarial purification against evasion attacks. arXiv preprint arXiv:2312.06423.
Zhou, Y., Tu, F., Sha, K., Ding, J., Chen, H., 2024. A survey on data quality dimensions and tools for machine learning invited paper. In: 2024 IEEE International Conference on Artificial Intelligence Testing. AITest, IEEE, pp. 120–131.



Faithful Chiagoziem Onwuegbuche is a PhD candidate in Machine Learning at the SFI Center for Research Training in Machine Learning (ML-Labs) at University College Dublin (UCD), Ireland. He is also affiliated with the SFI Research Centre for Software (LERO). His research interests lie at the intersection of artificial intelligence and blockchain technology, with a focus on their applications in finance and cybersecurity. His ultimate goal is to develop AI systems capable of securing critical infrastructure, detecting financial fraud, and promoting financial inclusion. Prior to his PhD, he earned two master's degrees: a Master of Science in Financial Technology (FinTech) with Distinction from the University of Stirling, UK, and a Master of Science in Financial Mathematics from the Pan African University, Kenya, as a Commonwealth Shared Scholar and African Union Scholar, respectively. He has professional experience as a data scientist, machine learning researcher, and lecturer. You can read more about him at: https://faithfulco.github.io/.



Sunday Adelodun holds a B.Sc. in Electrical and Electronic Engineering from the University of Ibadan, Nigeria. He is an Embedded Systems Engineer specialising in firmware development, embedded computing, and system architecture. His research interests include embedded systems design, IoT, and Edge AI, with a focus on optimising performance and efficiency in resource-constrained environments, including systems leveraging embedded operating systems.



Dr. Anca Jurcut is an Assistant Professor in the School of Computer Science, University College Dublin (UCD), Ireland, since 2015. She received a BSc in Computer Science and Mathematics from West University of Timisoara, Romania in 2007 and a PhD in Security Engineering from the University of Limerick (UL), Ireland in 2013 funded by the Irish Research Council for Science Engineering and Technology. She worked as a postdoctoral researcher at UL as a member of the Data Communication Security Laboratory and as a Software Engineer in IBM in Dublin, Ireland in the area of data security and formal verification. Dr. Jurcut research interests include Security Protocols Design and Analysis, Automated Techniques for Formal Verification, Network Security, Attack Detection and Prevention Techniques, Security for the Internet of Things, and Applications of Blockchain for Security and Privacy. Dr. Jurcut has several key contributions in research focusing on detection and prevention techniques of attacks over networks, the design and analysis of security protocols, automated techniques for formal verification, and security for mobile edge computing (MEC). More Info: https://people.ucd.ie/anca.jurcut.



Liliana Pasquale received her PhD from Politecnico di Milano (Italy) in 2011. She is an Associate Professor at University College Dublin (Ireland) and a funded investigator at Lero - the SFI Research Centre for Software. Her research interests include requirements engineering and adaptive systems, focusing on security, privacy, and digital forensics. She has served in the Program and Organising Committee of prestigious software engineering conferences, such as ICSE, FSE, ASE, RE. She is an associate editor of the IEEE TSE journal, department editor of the IEEE Security & Privacy Magazine and a member of the review board of the ACM TOSEM journal.